

ENCICLOPÉDIA VISUAL DE COMPUTAÇÃO

CIBERSEGURANÇA & ETHICAL HACKING

Criptografia clássica e moderna, OWASP Top 10, Red Team e Blue Team

METODOLOGIA TRÍPLICE

UNINTER (Base Acadêmica) + ALURA (Mercado) + FEYNMAN (Didática Visual)

VOLUME COM 71 MAPAS MENTAIS ESTRUTURADOS EM A4 PAISAGEM

SUMÁRIO DO VOLUME

#148 - CRIPTOGRAFIA SIMÉTRICA VS ASSIMÉTRICA	pág. 4	#372 - SIEM: CENTRALIZANDO EVENTOS DE SEGURANÇA COM SP	pág. 36
#341 - A TRÍADE CIA: OS 3 MANDAMENTOS DA SEGURANÇA	pág. 5	#373 - EDR VS ANTIVÍRUS TRADICIONAL	pág. 37
#342 - AMEAÇA, VULNERABILIDADE, EXPLOIT E RISCO	pág. 6	#374 - IDS E IPS (SNORT, SURICATA): DETECÇÃO E PREVENÇ	pág. 38
#343 - TIPOS DE HACKERS: WHITE, BLACK E GRAY HAT	pág. 7	#375 - WAF (WEB APPLICATION FIREWALL): CLOUDFLARE E AW	pág. 39
#344 - CRIPTOGRAFIA SIMÉTRICA (AES E CHACHA20)	pág. 8	#376 - HONEYPOTS: ARMADILHAS PARA ATRAIR E ESTUDAR INV	pág. 40
#345 - CRIPTOGRAFIA ASSIMÉTRICA (RSA E CURVAS ELÍPTICA	pág. 9	#377 - RESPOSTA A INCIDENTES: AS 6 FASES DO NIST	pág. 41
#346 - FUNÇÕES DE HASH CRIPTOGRÁFICO (SHA-256, SHA-3)	pág. 10	#378 - FORENSE DIGITAL E CADEIA DE CUSTÓDIA	pág. 42
#347 - ARMAZENAMENTO SEGURO DE SENHAS: SALT, PEPPER E	pág. 11	#379 - THREAT INTELLIGENCE E FRAMEWORK MITRE ATT&CK	pág. 43
#348 - ASSINATURAS DIGITAIS E INFRAESTRUTURA DE CHAVES	pág. 12	#380 - HARDENING DE SERVIDORES: MENOS PORTAS, MENOS RI	pág. 44
#349 - OWASP TOP 10: O PADRÃO OURO DE RISCOS EM APLICA	pág. 13	#381 - O QUE É DEVSECOPS? (SHIFT LEFT SECURITY)	pág. 45
#350 - SQL INJECTION (SQLi): ANATOMIA E DEFESA COM PRE	pág. 14	#382 - ANÁLISE ESTÁTICA DE CÓDIGO (SAST) COM SONARQUBE	pág. 46
#351 - CROSS-SITE SCRIPTING (XSS)	pág. 15	#383 - ANÁLISE DINÂMICA DE SEGURANÇA (DAST) COM OWASP	pág. 47
#352 - CROSS-SITE REQUEST FORGERY (CSRF) E TOKENS ANTI	pág. 16	#384 - SECRET SCANNING: EVITANDO VAZAMENTOS DE CHAVES	pág. 48
#353 - QUEBRA DE CONTROLE DE ACESSO (IDOR)	pág. 17	#385 - LGPD: PRINCÍPIOS E DIREITOS	pág. 49
#354 - FALHAS CRIPTOGRÁFICAS E EXPOSIÇÃO DE DADOS SENS	pág. 18	#386 - PRIVACY BY DESIGN E PRIVACY BY DEFAULT	pág. 50
#355 - FALHAS DE AUTENTICAÇÃO: BRUTE FORCE & CREDENTIAL	pág. 19	#387 - ARQUITETURA ZERO TRUST: NUNCA CONFIE, SEMPRE VE	pág. 51
#356 - AUTENTICAÇÃO MULTIFATOR (MFA/2FA) COM TOTP	pág. 20	#388 - SEGURANÇA EM NUVEM: RESPONSABILIDADE COMPARTILH	pág. 52
#357 - INSECURE DESERIALIZATION: EXECUÇÃO REMOTA VIA O	pág. 21	#389 - IAM: RBAC vs ABAC	pág. 53
#358 - SSRF (SERVER-SIDE REQUEST FORGERY)	pág. 22	#390 - PLANO DE CONTINUIDADE DE NEGÓCIOS E DISASTER RE	pág. 54
#359 - COMPONENTES VULNERÁVEIS E DESATUALIZADOS (SCA E	pág. 23	#391 - RANSOMWARE: ANATOMIA, DISSEMINAÇÃO E PREVENÇÃO	pág. 55
#360 - HEADERS DE SEGURANÇA HTTP: CSP, HSTS, X-FRAME-O	pág. 24	#392 - BOTNETS E DISPOSITIVOS IOT COMPROMETIDOS	pág. 56
#361 - FASES DE UM PENTEST PROFISSIONAL: PTES E RECONH	pág. 25	#393 - ATAQUES MAN-IN-THE-MIDDLE (MITM) E ARP SPOOFING	pág. 57
#362 - OSINT: COLETA DE INFORMAÇÕES PÚBLICAS	pág. 26	#394 - ATAQUES DE SUPPLY CHAIN (CADEIA DE SUPRIMENTOS)	pág. 58
#363 - VARREDURA DE REDES COM NMAP: PORT SCANNING E DE	pág. 27	#395 - ZERO-DAY EXPLOITS E O MERCADO DE VULNERABILIDAD	pág. 59
#364 - ANÁLISE DE TRÁFEGO DE REDE COM WIRESHARK E TCPD	pág. 28	#396 - ENGENHARIA REVERSA DE BINÁRIOS COM GHIDRA	pág. 60
#365 - INTERCEPÇÃO DE REQUISIÇÕES COM BURP SUITE	pág. 29	#397 - SEGURANÇA EM APIS: OWASP API SECURITY TOP 10	pág. 61
#366 - FUZZING DE DIRETÓRIOS COM GOBUSTER E FFUF	pág. 30	#398 - SEGURANÇA WI-FI: WPA2 VS WPA3	pág. 62
#367 - INTRODUÇÃO AO METASPLOIT FRAMEWORK	pág. 31	#399 - DEEPFAKES, IA MALICIOSA E CLONAGEM DE VOZ	pág. 63
#368 - ENGENHARIA SOCIAL E PHISHING SIMULADO	pág. 32	#400 - BUG BOUNTY: FATURANDO AO CAÇAR FALHAS LEGALMENT	pág. 64
#369 - MOVIMENTAÇÃO LATERAL E ESCALAÇÃO DE PRIVILÉGIOS	pág. 33	#401 - MONTANDO SEU LAB DE ESTUDOS COM KALI LINUX E ME	pág. 65
#370 - ATAQUES DE NEGAÇÃO DE SERVIÇO (DoS vs DDoS)	pág. 34	#402 - PLATAFORMAS DE TREINAMENTO: HACK THE BOX E TRYH	pág. 66
#371 - O QUE FAZ UM SOC (SECURITY OPERATIONS CENTER)?	pág. 35	#403 - CERTIFICAÇÕES DE CIBERSEGURANÇA: SECURITY+, CEH	pág. 67

SUMÁRIO DO VOLUME (CONTINUAÇÃO)

#404 - SEGURANÇA EM CONTÊINERES: TRIVY E DOCKER BENCH	pág. 68
#405 - SEGURANÇA EM KUBERNETES: POD SECURITY STANDARDS	pág. 69
#406 - SEGURANÇA EM AD E KERBEROS	pág. 70
#407 - PASS-THE-HASH E GOLDEN TICKET NO ACTIVE DIRECTO	pág. 71
#408 - AUTOMAÇÃO DE SEGURANÇA COM PYTHON (SCAPY E REQU	pág. 72
#409 - RELATÓRIO DE PENTEST: DA FALHA TÉCNICA AO PREJU	pág. 73
#410 - O FUTURO DA CIBERSEGURANÇA: CRIPTOGRAFIA PÓS-QU	pág. 74

IDEIA CENTRAL



- Simétrica usa 1 chave secreta igual para trancar e destrancar.
- Assimétrica usa 2 chaves diferentes: uma pública, uma privada.

RESUMO EXPRESSO



- 1 chave secreta → rápido e simples → bom para dados locais
- 2 chaves (pública/privada) → seguro na internet → seguro na comunicação

PEGADINHAS



- Achar que a chave pública também serve para destrancar (não serve).
- Confundir: assimétrica é mais segura para compartilhar, mas mais

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `encrypt(msg, chavePublica)` e `decrypt(msg, cha`
- Resultado imediato no terminal



COMO FUNCIONA CADA UMA

- Simétrica: rápida, mas as duas partes precisam combinar a mesma chave antes.
- Assimétrica: a chave pública tranca, só a privada destranca.



CÓDIGO

RSA gera 2 chaves matemáticas ligadas entre si.

```
encrypt(msg,  
chavePublica) e  
decrypt(msg,  
chavePrivada)
```



LINGUAGENS

- Simétrica: AES protege arquivos e conexões rápidas.
- Assimétrica (RSA): HTTPS, assinatura digital, PIX, e-mail seguro



FLUXO BÁSICO

- PASSO 1: Você recebe a chave pública de alguém.
- PASSO 2: Usa essa chave pública para trancar a mensagem.
- PASSO 3: Só a chave privada da pessoa abre a mensagem.



148. CRIPTOGRAFIA SIMETRICA VS ASSIMETRICA

É o jeito de embaralhar informação para só quem tem a chave certa conseguir ler.

IDEIA CENTRAL



- Pense num cofre de banco: só quem pode vê, nada é trocado, e abre quando precisa.

RESUMO EXPRESSO



- Confidencialidade → Integridade → Disponibilidade = Dado protegido

PEGADINHAS



- Achar que só senha forte já resume toda a segurança.
- Dica: sem Disponibilidade, dado seguro e correto não serve de nada.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `hash(arquivo)` confere se o conteúdo mudou.
- Resultado imediato no terminal



341. A TRÍADE CIA: OS 3 MANDAMENTOS DA SEGURANÇA

São as 3 regras de ouro que garantem que uma informação esteja protegida, correta e acessível.

CONFIDENCIALIDADE

- Só pessoas autorizadas podem ver a informação.
- Sistema esconde dados de quem não tem permissão.



INTEGRIDADE

Garante que o dado não foi alterado sem permissão.

`hash(arquivo)` confere se o conteúdo mudou.



LINGUAGENS

- Bancos, hospitais, redes sociais e sistemas de login.
- Ferramentas: criptografia, backups, firewalls, senhas.



FLUXO DA PROTEÇÃO

- PASSO 1: Dado é criado e classificado como sensível
- PASSO 2: Sistema aplica senha, criptografia e controle de acesso
- PASSO 3: Usuário autorizado acessa o dado íntegro e disponível



IDEIA CENTRAL



- Pense numa casa: fechadura quebrada, ladrão querendo entrar, chave mestra e o que ele rouba.

RESUMO EXPRESSO



- Brecha (vulnerabilidade) → Invasor usa arma (exploit) → Vira estrago real (risco)

PEGADINHAS



- Achar que vulnerabilidade e ameaça são a mesma coisa, não são.
- Dica de ouro: sem exploit funcionando, a vulnerabilidade pode

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: $\text{Risco} = \text{Ameaça} \times \text{Vulnerabilidade} \times \text{Impacto}$
- Resultado imediato no terminal



342. AMEAÇA, VULNERABILIDADE, EXPLOIT E RISCO

São 4 peças diferentes do mesmo quebra-cabeça de segurança: a brecha, o perigo, a arma e o estrago.

VULNERABILIDADE VS

- Vulnerabilidade é a fechadura quebrada, o defeito que já existe no sistema.
- Ameaça é o ladrão, alguém ou algo com potencial de causar dano.



FÓRMULA DO RISCO

Risco nasce da combinação dos outros três elementos.

$$\text{Risco} = \text{Ameaça} \times \text{Vulnerabilidade} \times \text{Impacto}$$



LINGUAGENS

- Times de segurança usam isso para priorizar o que corrigir primeiro.
- Aparece em relatórios de pentest, CVE e análises de risco.



FLUXO BÁSICO

- PASSO 1: A vulnerabilidade existe, a fechadura está frágil
- PASSO 2: A ameaça (invasor) usa um exploit para forçar a fechadura
- PASSO 3: O risco se concretiza como o prejuízo real causado



IDEIA CENTRAL



- A técnica é a mesma, o que muda é a intenção e a permissão para usá-la.

RESUMO EXPRESSO



- Permissão + boa intenção → White Hat → sistema protegido e legal.

PEGADINHAS



- Achar que 'só estava curioso' livra alguém de responsabilidade legal.
- Dica de ouro: sem contrato de autorização, é crime, não é ética.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



343. TIPOS DE HACKERS: WHITE, BLACK E GRAY HAT

Hacker é quem domina sistemas de computador, mas a cor do chapéu define se usa esse poder para proteger ou para prejudicar.

WHITE HAT

- É o hacker ético, contratado para achar falhas antes dos criminosos.
- Sempre tem autorização por escrito para testar o sistema.



BLACK HAT E GRAY HAT

Black Hat invade sem permissão para roubar ou destruir dados.
Gray Hat invade sem permissão, mas sem intenção de causar dano.



LINGUAGENS

- White Hat atua em pentests, bug bounty e auditorias de segurança.
- Ferramentas: Kali Linux, Metasploit, Burp Suite, sempre em ambiente autorizado.



FLUXO DE

- PASSO 1: Empresa autoriza formalmente o teste de invasão.
- PASSO 2: White Hat busca falhas simulando um ataque real.
- PASSO 3: Relatório é entregue e as brechas são corrigidas.



IDEIA CENTRAL



- Uma única chave secreta serve pra embaralhar e desembaralhar a informação.

RESUMO EXPRESSO



- Chave secreta →
Tranca o cofre →
Mesma chave destranca depois

PEGADINHAS



- Perder a chave secreta significa perder o acesso aos dados pra sempre.
- Nunca envie a chave junto com a mensagem cifrada pelo mesmo

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `cifra = AES.encrypt(mensagem, chave_secreta)`
- Resultado imediato no terminal



344. CRIPTOGRAFIA SIMETRICA (AES E CHACHA20)

É um cadeado onde a mesma chave secreta tranca e destranca o cofre de dados.

AES E CHACHA20

- AES embaralha os dados em blocos fixos, como fatiar e misturar cartas.
- ChaCha20 mistura os dados em um fluxo contínuo, mais leve pra celulares.



CÓDIGO

Você passa a mensagem e a chave, o algoritmo devolve texto embaralhado.

```
cifra =  
AES.encrypt(mensagem,  
chave_secreta)
```



LINGUAGENS

- Proteger arquivos, senhas salvas e conversas do WhatsApp.
- Usado em HTTPS, VPNs e bibliotecas como OpenSSL e libsodium.



FLUXO BÁSICO

- PASSO 1: Você cria uma chave secreta única.
- PASSO 2: O algoritmo usa a chave pra trancar (cifrar) os dados.
- PASSO 3: A mesma chave destranca (decifra) os dados no destino.



IDEIA CENTRAL



- É como uma caixa de correio: qualquer um coloca carta, só você tem a chave que abre.

RESUMO EXPRESSO



- Chave pública tranca
→ Chave privada abre
→ Comunicação segura garantida.

PEGADINHAS



- Achar que a chave pública precisa ficar em segredo, ela não precisa.
- Dica de ouro: se a chave privada escapar, todo o sistema fica quebrado.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `gerarChaves()` → `{publica, privada}`
- Resultado imediato no terminal



345. CRIPTOGRAFIA ASSIMETRICA (RSA E CURVAS ELIPTICAS ECC)

É um sistema com duas chaves diferentes: uma para trancar e outra para abrir a informação.

CHAVE PÚBLICA

- A chave pública é o cadeado: você espalha para todo mundo usar.
- A chave privada é a chave real: fica só na sua máquina, nunca é enviada.



CÓDIGO

Criptografa com a pública, só a privada correspondente descriptografa.

```
gerarChaves() →  
{publica, privada}
```



LINGUAGENS

- HTTPS do seu navegador, apps de mensagem e assinaturas digitais.
- Exemplos: RSA (bancos, servidores), ECC (Bitcoin, WhatsApp,



FLUXO BÁSICO

- PASSO 1: Você gera um par de chaves matematicamente ligadas
- PASSO 2: Envia a chave pública, guarda a privada em segredo
- PASSO 3: Só quem tem a privada consegue abrir o que foi trancado com a pública



IDEIA CENTRAL



- Pega qualquer dado e gera um código curto e único, como uma impressão digital.

RESUMO EXPRESSO



- Arquivo → Função Hash
→ Impressão digital
única e comprovável

PEGADINHAS



- Achar que dá para reverter o hash e descobrir o arquivo original.
- Dica: para checar integridade, compare o hash novo com o hash

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: hash =
sha256(arquivo.pdf) →
'3a7f...'
- Resultado imediato no terminal



346. FUNÇÕES DE HASH CRIPTOGRÁFICO (SHA-256, SHA-3)

É uma máquina que transforma qualquer arquivo numa impressão digital única e de mão única.

MÃO ÚNICA

- Dá para gerar o código a partir do arquivo, mas nunca o arquivo a partir do código.
- Qualquer mudança mínima gera um código totalmente diferente.



CÓDIGO

Sempre gera um tamanho fixo, não importa o tamanho do arquivo.

hash =
sha256(arquivo.pdf) →
'3a7f...'



LINGUAGENS

- Verificar se um download não foi corrompido ou alterado.
- Usado em senhas, blockchain e assinaturas digitais.



FLUXO BÁSICO

- PASSO 1: Você pega o arquivo original
- PASSO 2: A função de hash processa e comprime tudo
- PASSO 3: Sai uma sequência única que representa o arquivo



IDEIA CENTRAL



- Nunca guarde a senha, guarde uma 'impressão digital' irreversível dela.

RESUMO EXPRESSO



- Senha + Salt único → Bcrypt/Argon2 (lento) → Hash irreconhecível guardado no banco.

PEGADINHAS

- MD5 e SHA1 são rápidos demais, hackers testam bilhões de senhas por segundo.
- Sem salt, senhas iguais geram o mesmo hash e caem em Rainbow

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: Nunca faça isso: `hash = md5(senha)`.
- Resultado imediato no terminal



347. ARMAZENAMENTO SEGURO DE SENHAS: SALT, PEPPER E BCRYPT/ARGON2

É a arte de guardar senhas de um jeito que, mesmo se roubarem o banco de dados, ninguém descubra a senha real.

HASH, SALT E PEPPER

- Hash transforma a senha num código embaralhado sem volta.
- Salt é um tempero único por usuário, misturado antes do hash.
- Pepper é um segredo extra guardado fora do banco de dados.



CÓDIGO

Nunca faça isso: `hash = md5(senha)`.

Faça isso: `hash = bcrypt(senha + salt, custo=12)`.



LINGUAGENS

- Login de sites, apps bancários e redes sociais.
- Bibliotecas como bcrypt, Argon2, PBKDF2 em Node, Python, Java.



FLUXO BÁSICO

- PASSO 1: Usuário cria a senha no cadastro.
- PASSO 2: Sistema gera um salt único e mistura com a senha antes de aplicar Bcrypt/Argon2.
- PASSO 3: Só o hash final e o salt vão pro banco, a senha original é descartada.



IDEIA CENTRAL



- É como um selo de cera exclusivo que só você consegue fazer, mas todo mundo consegue verificar.

RESUMO EXPRESSO



- Chave privada assina → Chave pública confere → Autoria fica provada.

PEGADINHAS



- Confundir assinatura digital com senha: são coisas totalmente diferentes.
- Perder a chave privada é como perder sua identidade: proteja-a

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: assinatura = criptografar(hash(documento), ch
- Resultado imediato no terminal



348. ASSINATURAS DIGITAIS E INFRAESTRUTURA DE CHAVES PÚBLICAS (PKI)

É um jeito matemático de provar quem escreveu algo, sem margem para fingir ser outra pessoa.

CHAVE PÚBLICA

- Cada pessoa tem duas chaves: uma secreta (privada) e uma aberta (pública).
- O que uma chave fecha, só a outra chave do par consegue abrir ou confirmar.



COMO FUNCIONA

Você assina com a chave PRIVADA, todos verificam com a chave PÚBLICA.

```
assinatura =  
criptografar(hash(documento),  
chave_privada)
```



LINGUAGENS

- Certificados de site (HTTPS), assinatura de contratos e notas fiscais.
- Git commits assinados, apps de banco, certificado digital (ICP-)



FLUXO BÁSICO

- PASSO 1: O sistema gera um 'resumo único' (hash) do documento.
- PASSO 2: Esse resumo é assinado com a chave privada do autor.
- PASSO 3: Qualquer pessoa usa a chave pública para confirmar que bate.



IDEIA CENTRAL



- Um raio-x dos erros que hackers mais exploram em aplicações.

RESUMO EXPRESSO



- Falha comum → Lista prioriza risco → Time corrige antes do ataque

PEGADINHAS



- Achar que seguir a lista uma vez basta, ela muda com o tempo.
- Confundir a lista com um manual de código, ela é um guia de riscos.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `SELECT * FROM users WHERE nome = 'input_do_us`
- Resultado imediato no terminal



349. OWASP TOP 10: O PADRÃO OURO DE RISCOS EM APLICAÇÕES WEB

É uma lista das dez falhas de segurança mais perigosas que existem em sites e sistemas web.

QUEM CRIA A LISTA

- A OWASP é uma organização sem fins lucrativos de segurança.
- Atualiza o ranking a cada poucos anos com dados reais.



EXEMPLO DE FALHA

Injeção de código acontece quando dados do usuário não são filtrados.

```
SELECT * FROM users  
WHERE nome =  
'input_do_usuario'
```



LINGUAGENS

- Guia para times de desenvolvimento, QA e segurança da informação.
- Usado em testes de invasão (pentest) e auditorias.



COMO A LISTA FUNCIONA

- PASSO 1: Pesquisadores coletam falhas reais encontradas no mundo todo
- PASSO 2: As mais frequentes e graves são ranqueadas de 1 a 10
- PASSO 3: Times usam a lista como checklist de proteção



IDEIA CENTRAL



- Formulário confia demais no que o usuário digita e executa como comando.

RESUMO EXPRESSO



- Texto do usuário → Prepared Statement separa comando de dado → Ataque neutralizado.

PEGADINHAS



- Achar que só 'escapar aspas' já resolve o problema.
- Ouro: nunca monte SQL com + ou f-string, sempre use parâmetros.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `SELECT * FROM users WHERE nome = ' ' + input +`
- Resultado imediato no terminal



350. SQL INJECTION (SQLI): ANATOMIA E DEFESA COM PREPARED STATEMENTS

É quando um invasor digita código SQL num campo de texto para enganar o banco de dados.

COMO O

- O texto digitado é colado direto dentro da consulta SQL.
- O banco não sabe diferenciar 'dado' de 'comando'.



CÓDIGO

Errado: concatenar texto direto na query.

```
SELECT * FROM users  
WHERE nome = ' ' + input + ' '
```

Certo: usar placeholder e



LINGUAGENS

- Login, busca e cadastro em sites e apps.
- Prepared Statements existem em Java (JDBC), PHP (PDO), Python (psycopg2), Node (pg).



FLUXO BÁSICO

- PASSO 1: Usuário digita algo suspeito, tipo ' OR 1=1 --
- PASSO 2: Sistema separa comando fixo do dado (Prepared Statement).
- PASSO 3: Banco trata o texto só como valor, ataque não funciona.



IDEIA CENTRAL



- O site confia demais no que o usuário digita e acaba executando código estranho.

RESUMO EXPRESSO



- Usuário digita script → site exibe sem filtrar → navegador executa e rouba dados.

PEGADINHAS



- Achar que só sites 'feios' têm XSS, qualquer campo de texto é um risco.
- Dica de ouro: nunca confie no que o usuário digitou, sempre escape a saída.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código:



351. CROSS-SITE SCRIPTING (XSS)

É quando um site deixa um invasor injetar um script malicioso que roda no navegador de outra pessoa.

ONDE ACONTECE

- Caixas de comentário, busca, perfil de usuário e formulários sem filtro.
- Comum em apps feitos com PHP, JS puro, ou frameworks configurados errado.



COMO O ATAQUE FUNCIONA

- PASSO 1: Invasor envia um link ou comentário com script escondido
- PASSO 2: Vítima abre a página e o navegador executa o script sem suspeitar
- PASSO 3: Script rouba o cookie da vítima e envia para o invasor



IDEIA CENTRAL



- O golpista não rouba sua senha, ele usa sua sessão já logada contra você.

RESUMO EXPRESSO



- Sessão ativa → Requisição forjada → Token ausente bloqueia o ataque.

PEGADINHAS



- Achar que HTTPS por si só já impede CSRF, mas não impede.
- Dica de ouro: sempre valide o token no servidor, nunca confie só no cookie.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código:
- Resultado imediato no terminal



352. CROSS-SITE REQUEST FORGERY (CSRF) E TOKENS ANTI-CSRF

É quando um site malicioso faz seu navegador enviar uma ação sem você querer, usando seu login.

TRANSFERÊNCIA SEM

- Você abre um site falso enquanto está logado no banco em outra aba.
- O navegador envia seus cookies automaticamente, e a transferência é feita.



CÓDIGO

O servidor exige um token secreto extra em cada formulário sensível.



LINGUAGENS

- Bancos, e-commerces e painéis administrativos protegem formulários assim.
- Frameworks como Django, Rails e Spring já geram esse token.



FLUXO BÁSICO

- PASSO 1: Você loga no banco e recebe um cookie de sessão.
- PASSO 2: Um site malicioso cria um formulário escondido que aponta pro banco.
- PASSO 3: Sem token válido, o servidor bloqueia a transferência forjada.



IDEIA CENTRAL



- É como trocar o número do seu quarto de hotel e entrar no quarto de outra pessoa sem ninguém perguntar nada.

RESUMO EXPRESSO



- Trocar ID na URL → Sistema não verifica dono → Vazamento de dados de outro usuário

PEGADINHAS



- Achar que 'esconder' o ID no front-end já protege o dado.
- Regra de ouro: sempre valide a posse do recurso no back-end.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `if (pedido.id == usuarioLogado.id) permitir()`
- Resultado imediato no terminal



353. QUEBRA DE CONTROLE DE ACESSO (IDOR)

É quando o sistema esquece de checar se o dado pedido é realmente seu, e não do outro usuário.

IDOR

- O sistema usa um ID direto (ex: número da conta) sem verificar o dono.
- Trocar '123' por '124' na URL abre dados de outro usuário.



CÓDIGO

Falta comparar o ID da URL com o ID logado.

```
if (pedido.id ==  
    usuarioLogado.id)  
    permitir()
```



LINGUAGENS

- Comum em APIs REST e painéis com URLs como `/pedido?id=45`.
- Testado em pentests com Burp Suite e OWASP ZAP.



FLUXO BÁSICO

- PASSO 1: Usuário acessa sua própria página com um ID na URL
- PASSO 2: Ele muda o número do ID manualmente
- PASSO 3: Sistema entrega o dado sem checar o dono, e vaza informação



IDEIA CENTRAL



- Dado sensível sem cadeado forte é como carta aberta no correio, todo mundo lê.

RESUMO EXPRESSO



- Dado sensível exposto
→ Sem criptografia forte → Vazamento e roubo de informação

PEGADINHAS



- Usar MD5 ou SHA1 para senhas é usar cadeado velho que qualquer chave abre.
- Achar que 'só é teste' e esquecer a chave hardcoded no código

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `const senhaApi = '12345secreta'; // NUNCA faç`
- Resultado imediato no terminal



354. FALHAS CRIPTOGRÁFICAS E EXPOSIÇÃO DE DADOS SENSÍVEIS

É quando dados secretos (senhas, cartões) ficam desprotegidos ou mal disfarçados, e qualquer um consegue ler.

TRÁFEGO SEM HTTPS

- Sem HTTPS, dados viajam na internet como texto puro, sem envelope.
- Qualquer um na mesma rede pode 'escutar' a conversa.



CHAVES HARDCODED

Colocar senha ou chave direto no código é escrever a senha na porta de casa.

```
const senhaApi =  
'12345secreta'; // NUNCA  
faça isso
```



LINGUAGENS

- Login, pagamentos, apps de banco e qualquer troca de dados na web.
- TLS/HTTPS, cofres de segredos como Vault, AWS Secrets Manager.



FLUXO BÁSICO

- PASSO 1: Dado sensível é criado ou enviado pelo usuário
- PASSO 2: Sistema criptografa (ou falha em criptografar) o dado
- PASSO 3: Dado protegido chega seguro, ou exposto se algo falhou



IDEIA CENTRAL



- É como testar todas as chaves de um chaveiro até uma abrir a porta.

RESUMO EXPRESSO



- Muitas tentativas erradas → Sistema desconfia → Bloqueia e atrasa o invasor

PEGADINHAS



- Bloquear só por IP não funciona, atacante troca de IP facilmente.
- Ouro: combine CAPTCHA, MFA e limite por conta + por IP juntos

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `if (tentativas > 5) bloquear_por(60_segundos)`
- Resultado imediato no terminal



355. FALHAS DE AUTENTICAÇÃO: BRUTE FORCE & CREDENTIAL STUFFING

É quando um invasor tenta adivinhar senhas repetidamente até acertar e entrar num sistema.

TIPOS DE ATAQUE

- Brute Force: testa TODAS as combinações possíveis de senha.
- Ataque de Dicionário: testa só senhas comuns, tipo '123456' e 'senha123'.
- Credential Stuffing: usa senhas roubadas de outros sites.



RATE LIMITING

Limita quantas tentativas de login um IP ou usuário pode fazer por tempo.

```
if (tentativas > 5)
  bloquear_por(60_segundos)
```



LINGUAGENS

- Login de sites, apps bancários, painéis administrativos.
- Ferramentas: Fail2Ban, AWS WAF, Cloudflare, middlewares Express/Django



BLOQUEIO PROGRESSIVO

- PASSO 1: Usuário erra a senha algumas vezes seguidas
- PASSO 2: Sistema aumenta o tempo de espera a cada erro novo
- PASSO 3: Após muitas falhas, conta é bloqueada temporariamente



IDEIA CENTRAL



- Além da senha, você prova sua identidade com um código que expira rápido.

RESUMO EXPRESSO



- Segredo + Hora → Cálculo Matemático → Código de 6 dígitos temporário

PEGADINHAS



- Se o relógio do celular estiver desregulado, o código não bate.
- Perder o celular sem backup do segredo trava o acesso à conta.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `codigo = HASH(segredo + horario_atual_em_bloc)`
- Resultado imediato no terminal



356. AUTENTICAÇÃO MULTIFATOR (MFA/2FA) COM TOTP

É uma senha extra que muda a cada 30 segundos para provar que você é você mesmo.

TOTP

- Um 'segredo' secreto é combinado com a hora atual para gerar o código.
- Servidor e app calculam o mesmo código, sem precisar se comunicar.



CÓDIGO

```
codigo = HASH(segredo +  
horario_atual_em_blocos_de_30s)  
  
totp.generate(secretKey)
```



LINGUAGENS

- Login em bancos, e-mails e redes sociais.
- Apps: Google Authenticator, Authy, Microsoft Authenticator.



FLUXO BÁSICO

- PASSO 1: Você escaneia um QR Code que contém um segredo único
- PASSO 2: O app usa esse segredo + a hora atual para gerar um número de 6 dígitos
- PASSO 3: Você digita o número e o servidor confirma se ele bate com o que ele mesmo calculou



IDEIA CENTRAL



- Serializar é empacotar um objeto para enviar. Desserializar é abrir o pacote e usar de novo.

RESUMO EXPRESSO



- Dado não confiável → Desserialização cega → Execução remota de código

PEGADINHAS



- Achar que só arquivos '.exe' são perigosos, dados também podem ser.
- Usar pickle ou readObject em dados de usuários é abrir a

EXEMPLO PRÁTICO

- Aplicação prática:
- Código:
`pickle.loads(dados_da_internet)`
já é um risco
- Resultado imediato no terminal



SERIALIZAÇÃO

- Python transforma objetos em bytes usando 'pickle'.
- Java faz o mesmo com Serializable e ObjectOutputStream.



CÓDIGO PERIGOSO

`pickle.loads(dados_da_internet)` já é um risco grave.

`ObjectInputStream.readObject(dadosExternos)`

357. INSECURE DESERIALIZATION: EXECUÇÃO REMOTA VIA OBJETOS

É quando o sistema 'reconstrói' dados vindos de fora sem checar se são seguros, e isso permite rodar código malicioso.

ONDE ACONTECE

- Cookies de sessão, APIs, filas de mensagens e cache.
- Ferramentas: pickle, Java RMI, PHP unserialize.



COMO O ATAQUE FUNCIONA

- PASSO 1: Atacante cria um objeto malicioso disfarçado de dado comum
- PASSO 2: Sistema desserializa sem validar a origem
- PASSO 3: Código escondido no objeto executa no servidor



IDEIA CENTRAL



- O servidor vira um 'mensageiro' que o hacker controla para entregar cartas em lugares proibidos.

RESUMO EXPRESSO



- URL maliciosa → Servidor busca por você → Segredos internos expostos

PEGADINHAS



- Achar que bloquear só 'localhost' resolve (esquece IPv6, redirects, DNS).
- Esquecer que a nuvem tem um endereço de metadados tipo

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `fetch(url_do_usuario)` sem validar destino
- Resultado imediato no terminal



358. SSRF (SERVER-SIDE REQUEST FORGERY)

É enganar o servidor para ele mesmo fazer pedidos perigosos que o atacante não conseguiria fazer direto.

COMO O

- App aceita uma URL do usuário e busca o conteúdo dela sem checar.
- Servidor confia demais e visita endereços internos escondidos.



CÓDIGO

Campo de 'importar imagem por URL' é a porta de entrada clássica.

```
fetch(url_do_usuario)
sem validar destino
```



ONDE É USADO

- Roubar credenciais de metadados da AWS/Azure/GCP.
- Escanear e acessar sistemas internos que não têm IP público.



FLUXO BÁSICO

- PASSO 1: Atacante envia uma URL maliciosa no campo do app
- PASSO 2: Servidor faz a requisição em nome do atacante
- PASSO 3: Resposta interna (senha, token) volta pra tela do atacante



IDEIA CENTRAL



- Seu código é seguro, mas as peças que você baixou de outros podem não ser.

RESUMO EXPRESSO



- Biblioteca desatualizada
→ Falha conhecida
exposta → Porta aberta
para ataque

PEGADINHAS



- Achar que 'funciona' significa 'seguro' e nunca atualizar as libs.
- Ouro: rode o scanner de dependências toda semana, não só uma vez.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



359. COMPONENTES VULNERÁVEIS E DESATUALIZADOS (SCA E CVES)

É o risco de usar bibliotecas prontas que têm falhas conhecidas e ninguém corrigiu ainda.

SCA

- Ferramenta que escaneia seu projeto e lista todas as bibliotecas usadas.
- Compara cada uma com um banco de dados de vulnerabilidades conhecidas.



CVE

É o 'RG' de uma falha de segurança já catalogada publicamente.
npm audit / pip-audit / mvn dependency-check



LINGUAGENS

- Em qualquer projeto que usa pacotes externos (quase todos).
- Snyk, Dependabot, OWASP Dependency-Check, GitHub Security.



FLUXO BÁSICO

- PASSO 1: A ferramenta lista todas as dependências do projeto
- PASSO 2: Cruza cada versão com o banco de CVEs conhecidos
- PASSO 3: Alerta e sugere a versão corrigida para atualizar



IDEIA CENTRAL



- O servidor entrega o site e um 'manual de segurança' junto, no cabeçalho da resposta.

RESUMO EXPRESSO



- Servidor envia regra → Navegador obedece → Ataque é bloqueado antes de acontecer.

PEGADINHAS



- Esquecer o HSTS deixa a porta aberta para downgrade de HTTPS para HTTP.
- CSP mal configurada pode quebrar o próprio site, sempre teste

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



360. HEADERS DE SEGURANÇA HTTP: CSP, HSTS, X- FRAME- OPTIONS

São instruções extras que o servidor manda junto com o site, dizendo ao navegador como se proteger.

CSP

- Diz ao navegador de onde ele PODE carregar scripts, imagens e estilos.
- Bloqueia scripts maliciosos injetados por invasores (XSS).



CÓDIGO

Cada header é uma linha de texto simples enviada na resposta HTTP.
Content-Security-Policy:
default-src 'self'



LINGUAGENS

- Configurado no servidor: Nginx, Apache, Node/Express, Cloudflare.
- Toda aplicação web séria usa isso, de bancos a e-commerces



FLUXO BÁSICO

- PASSO 1: Navegador pede a página ao servidor
- PASSO 2: Servidor responde com o HTML + headers de segurança
- PASSO 3: Navegador lê os headers e aplica as regras antes de renderizar



IDEIA CENTRAL



- É como contratar um ladrão ético para testar as fechaduras da sua casa.

RESUMO EXPRESSO



- Contrato → Reconhecimento → Relatório Executivo Final

PEGADINHAS



- Testar algo fora do escopo pode ser crime, mesmo sem intenção.
- Dica de ouro: sem contrato assinado, nunca inicie o teste.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `escopo = alvos + regras + data_limite`
- Resultado imediato no terminal



361. FASES DE UM PENTEST PROFISSIONAL: PTES E RECONHECIMENTO

Pentest é simular um ataque de hacker de forma autorizada para encontrar falhas antes dos criminosos.

PTES: O

- PTES é o roteiro padrão que organiza todo pentest em 7 etapas.
- Garante que nada importante seja esquecido no teste.



ESCOPO E CONTRATO

Documento que diz o que PODE e o que NÃO PODE ser testado.

`escopo = alvos + regras + data_limite`



LINGUAGENS

- Empresas de segurança, bancos e times de TI corporativos.
- Ferramentas: Nmap, Shodan, Maltego, Recon-ng.



DO CONTRATO

- PASSO 1: Assinar o contrato de escopo com o cliente
- PASSO 2: Fazer o reconhecimento para mapear o alvo
- PASSO 3: Entregar o relatório executivo com riscos e soluções



IDEIA CENTRAL



- Como um detetive, você junta pistas espalhadas na web para montar um quadro completo.

RESUMO EXPRESSO



- Dado público disperso
→ Coleta organizada →
Mapa de riscos da empresa

362. OSINT: COLETA DE INFORMAÇÕES PUBLICAS

É a arte de encontrar informações valiosas que já estão públicas na internet, sem invadir nada.

TIPOS DE ALVO

- Funcionários: nomes, cargos e redes sociais da empresa.
- E-mails vazados: contas expostas em brechas antigas.



FERRAMENTAS OU COMANDO

Buscas avançadas filtram resultados específicos no Google.
site:empresa.com
filetype:pdf



LINGUAGENS

- Pentest e auditoria de segurança autorizada por empresas.
- Ferramentas: theHarvester, Maltego, Shodan, LinkedIn.



FLUXO BÁSICO

- PASSO 1: Definir o alvo autorizado (domínio ou empresa)
- PASSO 2: Buscar em redes sociais, sites e bancos de vazamentos
- PASSO 3: Organizar tudo num mapa de funcionários e tecnologias



PEGADINHAS



- Achar que 'público' significa 'permitido usar para qualquer fim'.
- Sempre tenha autorização antes de investigar uma empresa real

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



IDEIA CENTRAL



- Nmap manda pacotes de rede e escuta as respostas pra mapear máquinas e serviços.

RESUMO EXPRESSO



- Enviar pacote →
Analisar resposta →
Mapear portas, serviços e SO do alvo

PEGADINHAS



- Escanear rede sem autorização é crime, sempre peça permissão antes.
- Firewalls podem mascarar portas como filtradas, confundindo o

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



363. VARREDURA DE REDES COM NMAP: PORT SCANNING E DETECÇÃO DE SO

Nmap é uma ferramenta que bate na porta de cada computador da rede pra ver quem está em casa.

PORT SCANNING

- Cada porta é como uma porta numerada de um prédio, algumas trancadas e outras abertas.
- Nmap testa milhares de portas e classifica como aberta, fechada ou filtrada.



CÓDIGO

Comando básico define alvo e tipo de varredura.
`nmap -sV -O -p 1-1000 192.168.1.10`



LINGUAGENS

- Pentests, auditorias de segurança e administração de redes corporativas.
- Usado com Wireshark, Metasploit e scripts NSE em conjunto.



FLUXO BÁSICO

- PASSO 1: Nmap envia pacotes de teste pra cada porta do alvo
- PASSO 2: Analisa as respostas pra descobrir portas abertas e serviços rodando
- PASSO 3: Cruza dados de resposta com banco interno pra adivinhar o sistema operacional



IDEIA CENTRAL



- Toda comunicação na internet vira pacotes, e você pode espiar cada um deles.

RESUMO EXPRESSO



- Capturar pacote →
Decodificar protocolo
→ Investigar
comunicação

364. ANÁLISE DE TRÁFEGO DE REDE COM WIRESHARK E TCPDUMP

CAPTURA DE PACOTES

- A placa de rede entra em 'modo promíscuo' e escuta tudo que passa.
- Cada pacote capturado guarda cabeçalho, protocolo e dados.



FILTROS DE CAPTURA

Filtros escolhem só o tráfego que interessa, ignorando o resto.
`tcpdump -i eth0 port 80`



PEGADINHAS



- Capturar tráfego sem autorização é crime, sempre peça permissão.
- Tráfego criptografado (HTTPS) esconde o conteúdo, só mostra o envelope.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



É como abrir o envelope de cada carta que passa pela rede para ver o que realmente está sendo dito.

LINGUAGENS

- Investigar ataques, lentidão de rede e vazamento de dados.
- Ferramentas: Wireshark (interface gráfica) e tcpdump (linha de comando)



FLUXO BÁSICO

- PASSO 1: A interface de rede captura os pacotes que trafegam
- PASSO 2: A ferramenta organiza e decodifica cada protocolo (HTTP, DNS, TCP)
- PASSO 3: Você analisa os dados para encontrar problemas ou provas de invasão



IDEIA CENTRAL



- Burp Suite se coloca entre seu navegador e o site para ver tudo que passa.

RESUMO EXPRESSO



- Requisição sai → Burp intercepta e edita → Servidor recebe versão alterada

PEGADINHAS



- Esquecer o proxy ligado deixa a navegação toda travada.
- Sempre confirme autorização antes de testar um sistema real.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `price=100 → price=1`
- Resultado imediato no terminal



365. INTERCEPÇÃO DE REQUISIÇÕES COM BURP SUITE

É um 'carteiro espião' que para sua mensagem no meio do caminho antes de ela chegar ao servidor.

PROXY INTERCEPTADOR

- Navegador envia dados para o Burp em vez de ir direto ao servidor.
- Burp segura a requisição em pausa antes de liberar.



EDITANDO A REQUISIÇÃO

Você troca valores como se editasse um texto simples.

`price=100 → price=1`



LINGUAGENS

- Testes de segurança em apps web e APIs.
- Usado por pentesters e times de segurança ofensiva.



FLUXO BÁSICO

- PASSO 1: Navegador manda a requisição pro proxy do Burp
- PASSO 2: Burp congela e mostra pra você editar os dados
- PASSO 3: Você libera e a requisição alterada segue pro servidor



IDEIA CENTRAL



- É como bater em toda porta de um prédio pra ver quais existem, mesmo sem placa na frente.

RESUMO EXPRESSO



- Lista de palavras → Testa cada uma na URL → Encontra painel admin ou backup esquecido.

PEGADINHAS



- Rodar sem permissão do dono do site é crime, não é brincadeira de laboratório.
- Ignorar o código de status da resposta e achar que tudo existe.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



366. FUZZING DE DIRETÓRIOS COM GOBUSTER E FFUF

É testar centenas de nomes de pastas e arquivos em um site para achar o que está escondido, mas não apagado.

COMO A

- A ferramenta pega uma lista gigante de palavras (wordlist) e testa uma por uma na URL.
- Ela olha a resposta do servidor pra saber se a página existe ou não.



CÓDIGO

Gobuster: você escolhe o modo dir e a wordlist.

gobuster dir -u

http://site.com -w wordlist.txt
ffuf usa FUZZ no lugar da parte que muda.

ffuf -u http://site.com/FUZZ



LINGUAGENS

- Testes de invasão (pentest) autorizados em sites e sistemas.
- Ferramentas: Gobuster, ffuf, dirb, wfuzz.



FLUXO BÁSICO

- PASSO 1: Você escolhe uma lista de nomes comuns (admin, backup, login).
- PASSO 2: A ferramenta testa cada nome na URL do site, um por um.
- PASSO 3: Ela mostra quais respostas deram 'existe' e quais deram 'não existe'.



IDEIA CENTRAL



- Metasploit é como um kit de chaves para testar se portas (falhas) estão trancadas de verdade.

RESUMO EXPRESSO



- Exploit encontra a falha
→ Payload entra →
Meterpreter dá o controle

PEGADINHAS



- Esquecer de configurar o IP certo no LHOST trava a conexão.
- Dica de ouro: sempre confirme autorização antes de testar qualquer alvo.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: Dentro do msfconsole você escolhe o módulo e
- Resultado imediato no terminal



367. INTRODUÇÃO AO METASPLOIT FRAMEWORK

É uma caixa de ferramentas usada por profissionais de segurança para testar e explorar falhas em sistemas de forma controlada.

EXPLOITS E PAYLOADS

- Exploit é a chave que aproveita uma falha específica no sistema alvo.
- Payload é o que roda depois que a porta é aberta, tipo um controle remoto.



CÓDIGO

Dentro do msfconsole você escolhe o módulo e configura opções.

use

exploit/windows/smb/ms17_010_eternalblue

LINGUAGENS

- Pentests autorizados e laboratórios de estudo como HackTheBox.
- Ferramentas: msfconsole, msfvenom, Armitage.



FLUXO BÁSICO

- PASSO 1: Escolhe o exploit certo pra falha do alvo
- PASSO 2: Liga o listener pra esperar a conexão de volta
- PASSO 3: Sessão Meterpreter abre e você controla a máquina



IDEIA CENTRAL



- O hacker ataca a mente humana, não o código do sistema.

RESUMO EXPRESSO



- Emoção manipulada → Clique impulsivo → Dados roubados

PEGADINHAS



- Achar que só gente 'burra' cai nesse tipo de golpe.
- Dica de ouro: sempre verifique o link antes de clicar.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



368. ENGENHARIA SOCIAL E PHISHING SIMULADO

É a arte de enganar pessoas, não máquinas, para roubar informações ou acesso.

PSICOLOGIA HUMANA

- Usa medo, urgência e curiosidade para te fazer agir sem pensar.
- Confiança e autoridade fazem você baixar a guarda.



ANATOMIA DO

Remetente parecido, mas com erro sutil no domínio. de: suporte@banc0-seguro.com



LINGUAGENS

- Treinamentos corporativos de segurança usam phishing simulado.
- Ferramentas como GoPhish e KnowBe4 testam funcionários.



FLUXO DO ATAQUE

- PASSO 1: O atacante cria um gancho emocional convincente
- PASSO 2: A vítima clica no link ou responde sem verificar
- PASSO 3: Dados ou acesso são roubados silenciosamente



IDEIA CENTRAL



- É como um zelador achando a chave-mestra do prédio e virando o síndico.

RESUMO EXPRESSO



- Usuário comum → Explora falha de configuração → Vira root/Administrator

PEGADINHAS



- Achar que só senha forte resolve, ignorando permissões erradas.
- Dica de ouro: sempre revisar quem pode acessar o quê.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `sudo -l (Linux) / whoami /priv (Windows)`
- Resultado imediato no terminal



NÍVEIS DE PODER

- No Linux o chefe é 'root', no Windows é 'Administrator'.
- Usuário comum só abre poucas portas do sistema.



SINAIS DE FALHA

Comando revela permissões abertas por engano.

```
sudo -l (Linux) / whoami /priv (Windows)
```



LINGUAGENS

- Testes de segurança autorizados (pentest) e CTFs.
- Ferramentas de análise: LinPEAS, WinPEAS, BloodHound.



FLUXO BÁSICO

- PASSO 1: Invasor entra como usuário comum e comum apenas.
- PASSO 2: Ele procura arquivo, serviço ou senha mal configurada.
- PASSO 3: Usa a brecha e assume controle total (root/Admin).



369. MOVIMENTAÇÃO LATERAL E ESCALAÇÃO DE PRIVILEGIOS

É quando o invasor, já dentro do sistema como usuário comum, encontra brechas até virar o 'chefe' da máquina.

IDEIA CENTRAL



- Imagine uma loja com 1 caixa e 1000 pessoas fingindo comprar só para travar a fila.

RESUMO EXPRESSO



- Tráfego falso em massa
→ Recursos do servidor esgotam → Serviço fica indisponível

PEGADINHAS



- Confundir DDoS com invasão: aqui não roubam dados, só travam o serviço.
- Achar que é só volume: ataque de camada 7 (aplicação) usa poucos

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



370. ATAQUES DE NEGAÇÃO DE SERVIÇO (DOS VS DDOS)

É lotar um serviço de pedidos falsos até ele travar e parar de atender pedidos reais.

DOS VS DDOS

- DoS: um único atacante, um único computador atacando.
- DDoS: um exército de computadores infectados (botnet) atacando juntos.



SYN FLOOD E UDP FLOOD

SYN Flood: começa milhares de 'apertos de mão' TCP e nunca termina nenhum.
UDP Flood: envia lixo de dados UDP para entupir a banda da rede.



ONDE ISSO ACONTECE

- Servidores web, jogos online, bancos e provedores de internet.
- Ferramentas de defesa: firewalls, WAF, Cloudflare, AWS Shield.



FLUXO DO ATAQUE

- PASSO 1: Atacante controla vários dispositivos infectados (botnet)
- PASSO 2: Todos enviam tráfego falso ao mesmo tempo para o alvo
- PASSO 3: Servidor esgota memória, banda ou processamento e cai



IDEIA CENTRAL



- Um time que fica de olho na 'saúde digital' da empresa sem parar.

RESUMO EXPRESSO



- Vigilância constante → Detecção rápida → Empresa protegida

PEGADINHAS



- Achar que SOC só reage, mas ele também previne ataques.
- Ignorar alertas pequenos pode virar um problema gigante.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



371. O QUE FAZ UM SOC (SECURITY OPERATIONS CENTER)?

É a torre de controle que vigia os sistemas de uma empresa 24 horas por dia contra ataques digitais.

TORRE DE CONTROLE

- Telas mostram tudo que acontece na rede em tempo real.
- Cada movimento estranho gera um alerta automático.



ALERTA OU REGRA

Sistema compara ações com padrões normais e suspeitos.
SE login estranho ENTÃO dispara alarme.



LINGUAGENS

- Bancos, hospitais e empresas de tecnologia usam SOC.
- Ferramentas comuns: SIEM, firewall, antivírus corporativo.



FLUXO BÁSICO

- PASSO 1: Sensores captam tudo que passa pela rede
- PASSO 2: Analistas investigam o que parece perigoso
- PASSO 3: Time age rápido para bloquear a ameaça



IDEIA CENTRAL



- É como uma central de câmeras que junta vídeos de toda a casa num só monitor.

RESUMO EXPRESSO



- Logs espalhados → Centralização e correlação → Detecção de invasão

PEGADINHAS



- Achar que só coletar logs já resume a segurança da empresa.
- Dica: sem regras de correlação, o SIEM só acumula dados inúteis.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `index=firewall OR index=server login_failed >`
- Resultado imediato no terminal



372. SIEM: CENTRALIZANDO EVENTOS DE SEGURANÇA COM SPLUNK / ELASTIC

SIEM é um sistema que junta todos os registros de segurança da empresa num só lugar para detectar ataques.

COMO O

- Coleta logs de firewalls, servidores e bancos de dados em tempo real.
- Guarda tudo organizado para buscar e comparar depois.



CÓDIGO

Regras de correlação cruzam eventos de fontes diferentes.

```
index=firewall OR  
index=server  
login_failed > 5
```



LINGUAGENS

- Times de segurança (SOC) monitoram ataques 24 horas por dia.
- Ferramentas: Splunk, Elastic Stack (ELK), Wazuh.



FLUXO BÁSICO

- PASSO 1: Firewall, servidor e banco enviam logs para o SIEM
- PASSO 2: O sistema correlaciona eventos suspeitos entre as fontes
- PASSO 3: Se o padrão bater com invasão, gera um alerta



IDEIA CENTRAL



- Antivírus é vigia com álbum de fotos de bandidos conhecidos.
- EDR é detetive que observa ações suspeitas de qualquer um.

RESUMO EXPRESSO



- Comportamento estranho → EDR investiga ação → Ameaça nova é pega antes do estrago

PEGADINHAS



- Achar que antivírus tradicional pega tudo, inclusive malware novo.
- Ouro: malware inédito não tem assinatura, só comportamento denuncia

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: Regra tipo: SE processo.abrir_arquivo == 'ass
- Resultado imediato no terminal



373. EDR VS ANTIVÍRUS TRADICIONAL

Antivírus procura vilões conhecidos numa lista, EDR observa o comportamento de todos os processos para pegar vilões novos.

COMO FUNCIONA

- Antivírus compara arquivos com uma lista de assinaturas ruins.
- EDR grava e analisa o comportamento dos processos em tempo real.



SINAL DE ALERTA

Regra tipo: SE processo.abrir_arquivo == 'assinatura_conhecida'.

EDR usa: SE processo.criptografar_muitos_arquivos_rapi
== suspeito.

LINGUAGENS

- Empresas usam EDR para proteger notebooks e servidores.
- Ferramentas: CrowdStrike, SentinelOne, Microsoft Defender for Endpoint.



FLUXO DE DETECÇÃO

- PASSO 1: Processo desconhecido começa a agir no computador
- PASSO 2: EDR observa o comportamento, não o nome do arquivo
- PASSO 3: Ação suspeita é bloqueada e time de segurança é avisado



IDEIA CENTRAL



- IDS é o vigia que só avisa quando vê algo estranho na rede.
- IPS é o segurança que já bloqueia a ameaça na hora.

RESUMO EXPRESSO



- Pacote suspeito → Sistema analisa → Alerta ou bloqueio

PEGADINHAS



- Confundir IDS (só detecta) com IPS (detecta e bloqueia).
- Regras mal ajustadas geram muitos falsos alarmes.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `alert tcp any any -> any 80 (msg:"Ataque!"; s`
- Resultado imediato no terminal



374. IDS E IPS (SNORT, SURICATÁ): DETECÇÃO E PREVENÇÃO DE INTRUSÃO

São sistemas que vigiam o tráfego da rede e avisam ou bloqueiam conexões suspeitas, como um radar de segurança.

SNORT E SURICATÁ

- São programas famosos que leem cada pacote de dados da rede.
- Comparam o tráfego com uma lista de padrões de ataques conhecidos.



REGRA DE DETECÇÃO

Regras dizem o que é suspeito, tipo uma receita de alerta.

```
alert tcp any any -> any
80 (msg:"Ataque!";
sid:1000001;)
```



LINGUAGENS

- Protege servidores, redes corporativas e data centers.
- Ferramentas: Snort, Suricata, Zeek, Wireshark para análise.



FLUXO BÁSICO

- PASSO 1: O tráfego de rede passa pelo sensor do IDS/IPS
- PASSO 2: O sistema compara os pacotes com regras de ataques
- PASSO 3: Se achar ameaça, avisa (IDS) ou bloqueia (IPS)



IDEIA CENTRAL



- WAF é um filtro que fica na frente do seu site checando cada requisição.

RESUMO EXPRESSO



- Requisição chega → WAF inspeciona → Ataque bloqueado antes da API.

PEGADINHAS



- Achar que WAF resolve tudo e esquecer de corrigir o código vulnerável.
- Dica: configure regras customizadas, não confie só no padrão.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `if (input contém "'" OR 1=1") então BLOCK`
- Resultado imediato no terminal



375. WAF (WEB APPLICATION FIREWALL): CLOUDFLARE E AWS WAF

É um segurança na porta que examina cada visitante e bloqueia ataques antes de chegarem na sua API.

COMO FUNCIONA

- Analisa URL, headers e corpo da requisição em busca de padrões maliciosos.
- Compara com regras conhecidas de SQLi, XSS e bots.



REGRA (WAF RULE)

Regra diz: se detectar padrão suspeito, bloqueia ou desafia.

```
if (input contém "'" OR 1=1") então BLOCK
```



LINGUAGENS

- Protege sites, APIs REST e aplicações web expostas na internet.
- Ferramentas: Cloudflare WAF, AWS WAF, ModSecurity.



FLUXO BÁSICO

- PASSO 1: Usuário envia uma requisição para o site.
- PASSO 2: WAF inspeciona o conteúdo antes de deixar passar.
- PASSO 3: Se for ataque, bloqueia; se for seguro, libera para a API.



IDEIA CENTRAL



- É uma isca digital: parece um servidor real, mas é só uma armadilha vigiada.

RESUMO EXPRESSO



- Isca falsa → invasor cai na armadilha → dados do ataque são estudados

PEGADINHAS



- Achar que honeypot substitui firewall ou antivírus, o que é falso.
- Isolar mal o honeypot pode deixar ele virar porta de entrada real.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



376. HONEYPOTS: ARMADILHAS PARA ATRAIR E ESTUDAR INVASORES

Honeypot é um sistema fingindo ser um alvo real para atrair e estudar invasores.

COMO FUNCIONA

- Simula portas, arquivos e serviços falsos para parecer atraente ao invasor.
- Toda ação dentro dele é gravada em detalhes, sem risco ao sistema real.



TIPOS DE HONEYPOT

Baixa interação: simula poucos serviços, mais seguro e simples.
Alta interação: sistema quase real, captura mais dados do ataque.



LINGUAGENS

- Empresas de segurança e times de resposta a incidentes.
- Ferramentas como Honeyd, Cowrie e T-Pot.



FLUXO BÁSICO

- PASSO 1: Servidor falso é criado e exposto na rede
- PASSO 2: Invasor encontra e ataca o honeypot pensando ser real
- PASSO 3: Time de segurança analisa os dados coletados sobre o ataque



IDEIA CENTRAL



- É como um plano de incêndio: todo mundo sabe o que fazer antes do fogo começar.

RESUMO EXPRESSO



- Detectar → Isolar → Limpar → Restaurar → Aprender

PEGADINHAS



- Pular a fase de Lições Aprendidas e repetir o mesmo erro depois.
- Confundir Erradicação com Recuperação, são etapas diferentes.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `isolar_maquina()` → depois `investigar_causa()`
- Resultado imediato no terminal



377. RESPOSTA A INCIDENTES: AS 6 FASES DO NIST

É o passo a passo que uma equipe segue para lidar com um ataque, do primeiro sinal até o aprendizado final.

AS 6 FASES DO NIST

- Preparação, Identificação, Contenção, Erradicação, Recuperação e Lições Aprendidas.
- Cada fase só começa quando a anterior termina bem.



REGRA DE ORO

Contenção não é excluir o problema, é isolar antes.

```
isolar_maquina() →  
depois  
investigar_causa()
```



LINGUAGENS

- Times de SOC (Security Operations Center) em empresas.
- Ferramentas como SIEM, EDR e playbooks de resposta.



FLUXO DAS 6 FASES

- PASSO 1: Preparação - montar plano, ferramentas e equipe antes do ataque
- PASSO 2: Identificação e Contenção - detectar o incidente e isolar o dano
- PASSO 3: Erradicação, Recuperação e Lições Aprendidas - remover, restaurar e melhorar



IDEIA CENTRAL



- É como embalar uma prova de crime: se violar o lacre, ela perde valor no julgamento.

RESUMO EXPRESSO



- Coletar com cuidado → Provar que não mudou → Prova aceita na justiça

PEGADINHAS



- Analisar o dispositivo original destrói a prova para o tribunal.
- Dica de ouro: documente cada passo, sem registro não existe prova

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



378. FORENSE DIGITAL E CADEIA DE CUSTODIA

É o processo de coletar e proteger provas digitais para que elas continuem válidas e confiáveis num tribunal.

CADEIA DE CUSTÓDIA

- Registro de quem tocou na evidência, quando e por quê, do início ao fim.
- Qualquer falha nesse histórico pode anular a prova.



HASH: A

Um código único gerado a partir do arquivo, prova que nada foi alterado.
sha256sum evidencia.img



LINGUAGENS

- Investigação de crimes, perícia policial e casos judiciais.
- Ferramentas: Autopsy, FTK, EnCase.



FLUXO DA PERÍCIA

- PASSO 1: Isolar o dispositivo e gerar uma cópia exata (imagem forense)
- PASSO 2: Calcular o hash da cópia para comprovar integridade
- PASSO 3: Analisar a cópia, nunca o original, e documentar tudo



IDEIA CENTRAL



- Um mapa que mostra os truques que criminosos usam para invadir sistemas.

RESUMO EXPRESSO



- Ataque real → Mapeado no catálogo → Defesa mais rápida e precisa

PEGADINHAS



- Achar que o ATT&CK é uma ferramenta, na verdade é um catálogo de referência.
- Dica: comece pelas táticas antes de estudar cada técnica

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: T1078 (usar conta legítima roubada)
- Resultado imediato no terminal



379. THREAT INTELLIGENCE E FRAMEWORK MITRE ATT&CK

É um catálogo global que organiza como os hackers atacam, do início ao fim de uma invasão.

TÁTICAS, TÉCNICAS

- Tática é o objetivo do atacante, tipo roubar senha.
- Técnica é o método usado, procedimento é o passo exato aplicado.



ESTRUTURA DA MATRIZ

Cada coluna é uma tática, cada célula é uma técnica.

T1078 (usar conta legítima roubada)



LINGUAGENS

- Times de segurança usam para caçar ameaças e treinar defesas.
- Ferramentas como SIEM, EDR e relatórios de threat intel.



FLUXO BÁSICO

- PASSO 1: Atacante escolhe uma tática, como acesso inicial
- PASSO 2: Aplica uma técnica, como phishing por e-mail
- PASSO 3: Time de defesa identifica o padrão e bloqueia



IDEIA CENTRAL



- Cada serviço ligado é uma porta destrancada na sua casa.

RESUMO EXPRESSO



- Serviço parado → Porta fechada → Menos chance de invasão

PEGADINHAS



- Desligar serviço sem saber pra que serve pode travar o sistema.
- Dica: teste em ambiente de homologação antes de aplicar em produção.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



380. HARDENING DE SERVIDORES: MENOS PORTAS, MENOS RISCO

Hardening é deixar o servidor só com o essencial ligado, fechando tudo que não é usado.

SUPERFÍCIE DE ATAQUE

- É o total de pontos que um invasor pode tentar usar.
- Quanto menos serviços rodando, menos portas abertas.



COMANDOS ÚTEIS

Linux desliga serviço:
systemctl disable --now nome
Windows fecha porta:
New-NetFirewallRule - Action Block



LINGUAGENS

- Servidores web, bancos de dados e nuvem (AWS, Azure).
- Ferramentas: firewalld, iptables, Windows Defender Firewall.



FLUXO BÁSICO

- PASSO 1: Liste todos os serviços e portas ativas no servidor
- PASSO 2: Identifique o que realmente é usado pela aplicação
- PASSO 3: Desative serviços extras e bloqueie portas não usadas



IDEIA CENTRAL



- Segurança não é etapa final, é responsabilidade de todo mundo, desde o início.

RESUMO EXPRESSO



- Commit → Scan automático de segurança → Código seguro liberado antes de virar problema

PEGADINHAS



- Achar que segurança é só trabalho do time de segurança.
- Ignorar alertas do scanner porque 'depois eu corrijo'.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



381. O QUE É DEVSECOPS? (SHIFT LEFT SECURITY)

É colocar a segurança dentro do processo de criar software, desde o primeiro commit, e não só no final.

SHIFT LEFT

- Move a checagem de segurança para o começo do desenvolvimento, não pro final.
- Achar erro cedo custa menos tempo e dinheiro que corrigir depois.



CÓDIGO

Ferramentas de scan rodam automaticamente ao salvar o código.
git commit -> scan automático -> alerta se achar falha



LINGUAGENS

- Pipelines de CI/CD como GitHub Actions, GitLab CI, Jenkins.
- Ferramentas: SAST, DAST, SCA, Snyk, SonarQube.



FLUXO BÁSICO

- PASSO 1: Programador escreve e envia o código (commit)
- PASSO 2: Pipeline roda testes de segurança automáticos antes do deploy
- PASSO 3: Se achar risco, bloqueia; se estiver seguro, libera o código



IDEIA CENTRAL



- Encontrar problemas no código só de lê-lo, como um professor corrigindo uma redação.

RESUMO EXPRESSO



- Código escrito → SonarQube analisa sem executar → Relatório de falhas de segurança

PEGADINHAS



- Achar que passar no SonarQube significa código 100% seguro.
- Ignorar falsos positivos sem analisar o contexto real.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `sonar-scanner -Dsonar.projectKey=meu-app`
- Resultado imediato no terminal



382. ANALISE ESTÁTICA DE CÓDIGO (SAST) COM SONARQUBE

É como um revisor de texto que lê seu código à procura de erros e riscos, sem precisar rodar o programa.

SAST

- Sigla para Static Application Security Testing, teste de segurança estático.
- Analisa o código-fonte parado, sem executar nada.



REGRAS E QUALITY GATE

SonarQube compara seu código com regras prontas de boas práticas.

```
sonar-scanner -  
Dsonar.projectKey=meu-  
app
```



LINGUAGENS

- Integrado em pipelines de CI/CD antes do deploy.
- Usado com Java, JavaScript, Python, C#, entre outras.



FLUXO BÁSICO

- PASSO 1: Código é enviado para o SonarQube analisar
- PASSO 2: Ferramenta varre linha por linha buscando falhas conhecidas
- PASSO 3: Gera relatório com bugs, vulnerabilidades e code smells



IDEIA CENTRAL



- Testar o app enquanto ele está funcionando, não só o código parado.

RESUMO EXPRESSO



- App no ar → ZAP ataca de fora → Relatório de brechas reais

PEGADINHAS



- Rodar o ataque em produção sem avisar e derrubar o site.
- Ignorar falsos positivos e perder tempo caçando fantasma.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



383. ANÁLISE DINÂMICA DE SEGURANÇA (DAST) COM OWASP ZAP

É atacar sua própria aplicação rodando, como um ladrão testaria as fechaduras de uma casa.

COMO O ZAP FUNCIONA

- Ele navega pelo site sozinho, clicando em tudo (crawling).
- Depois envia dados maliciosos em cada campo para ver o que quebra.



CÓDIGO

Rodar via linha de comando no pipeline de CI/CD.
zap-baseline.py -t https://meusite.com



LINGUAGENS

- Antes de colocar o site no ar (ambiente de staging).
- Integrado no GitHub Actions, GitLab CI ou Jenkins.



FLUXO BÁSICO

- PASSO 1: ZAP mapeia todas as páginas e formulários do site
- PASSO 2: Ele ataca cada campo com SQL Injection, XSS e outros truques
- PASSO 3: Gera um relatório listando as portas destrancadas



IDEIA CENTRAL



- Imagine deixar a chave de casa dentro de uma caixa pública sem querer.
- O sistema avisa antes que isso vire um problema real.

RESUMO EXPRESSO



- Chave no código → Scanner detecta → Alerta ou bloqueio evita vazamento

PEGADINHAS



- Achar que apagar o arquivo depois resolve, o histórico ainda guarda a chave.
- Dica de ouro: use arquivos .env e adicione ele no .gitignore.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `API_KEY=abc123` (isso NUNCA deve ir pro GitHub)
- Resultado imediato no terminal



384. SECRET SCANNING: EVITANDO VAZAMENTOS DE CHAVES NO GITHUB

É um vigia automático que verifica seu código antes e depois do commit para achar senhas e chaves esquecidas.

SECRET SCANNING

- O GitHub varre cada commit procurando padrões de senhas e tokens.
- Se achar algo suspeito, ele alerta o dono do repositório na hora.



CÓDIGO

Nunca escreva senhas direto no código, use variáveis de ambiente.

```
API_KEY=abc123 (isso NUNCA deve ir pro GitHub)
```



LINGUAGENS

- Em qualquer repositório público ou privado no GitHub.
- Ferramentas: GitHub Secret Scanning, Gitleaks, TruffleHog.



FLUXO BÁSICO

- PASSO 1: Você tenta enviar (commit) um arquivo com uma chave secreta
- PASSO 2: A ferramenta compara o texto com padrões conhecidos de senhas
- PASSO 3: Se encontrar, bloqueia o envio ou dispara um alerta imediato



IDEIA CENTRAL



- Seus dados são seus, empresas precisam pedir permissão para usá-los.

RESUMO EXPRESSO



- Consentimento → Uso com finalidade → Descarte ou multa da ANPD

PEGADINHAS



- Achar que aceitar termos genéricos vale para qualquer uso futuro.
- Guardar dados 'para sempre' gera multa da ANPD

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `coletar(dado, motivo='cadastro') → proibido u`
- Resultado imediato no terminal



385. LGPD: PRINCÍPIOS E DIREITOS

Lei que protege os dados pessoais das pessoas e obriga empresas a usá-los com responsabilidade.

CONSENTIMENTO

- Usuário precisa autorizar de forma clara antes da coleta de dados.
- Consentimento pode ser revogado a qualquer momento pelo titular.



FINALIDADE

Dados só podem ser usados para o motivo informado ao usuário.

```
coletar(dado,
motivo='cadastro') →
proibido usar para outro fim
```



LINGUAGENS

- Sites, apps, e-commerces e sistemas que guardam dados de clientes.
- Formulários de cadastro, cookies, contratos e políticas de privacidade.



FLUXO DO DADO

- PASSO 1: Empresa pede consentimento com finalidade clara
- PASSO 2: Dado é usado só para o que foi combinado
- PASSO 3: Dado é descartado quando não é mais necessário



IDEIA CENTRAL



- Privacidade não é um remendo, é um tijolo da fundação do software.

RESUMO EXPRESSO



- Planejar cedo → Coletar o mínimo → Usuário protegido desde o início

PEGADINHAS



- Achar que privacidade é só termo de uso, e sim é código e arquitetura.
- Deixar compartilhamento de dados ligado por padrão é erro grave.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código:
`usuario.compartilharLocalizacao = false // pa`
- Resultado imediato no terminal



386. PRIVACY BY DESIGN E PRIVACY BY DEFAULT

É construir a proteção dos dados do usuário desde o primeiro rascunho do sistema, não depois.

BY DESIGN

- By Design: pensar em privacidade em cada decisão de arquitetura.
- By Default: o app já nasce configurado no modo mais privado possível.



REGRA NA PRÁTICA

Coletar só o dado essencial, nunca 'tudo que der'.

```
usuario.compartilharLocalizacao  
= false // padrão inicial
```



LINGUAGENS

- Cadastros, apps de saúde, redes sociais, sistemas bancários.
- Base de leis como LGPD e GDPR.



FLUXO BÁSICO

- PASSO 1: Antes de codar, mapeia-se quais dados realmente são necessários.
- PASSO 2: O sistema é desenhado com privacidade e criptografia embutidas.
- PASSO 3: O usuário recebe as opções mais privadas já ativadas de início



IDEIA CENTRAL



- Imagine um prédio onde todo mundo precisa mostrar crachá em cada porta, até para quem já entrou.

RESUMO EXPRESSO



- Pedido de acesso → Verificação constante → Liberação mínima e temporária

PEGADINHAS



- Achar que 'já estar logado' significa acesso permanente a tudo.
- Dica de ouro: sempre valide de novo, mesmo que a pessoa já esteja dentro

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `if (identidade_valida && permissao_ok) { libe`
- Resultado imediato no terminal



387. ARQUITETURA ZERO TRUST: NUNCA CONFIE, SEMPRE VERIFIQUE

É um modelo de segurança onde nenhum usuário ou dispositivo é confiável, mesmo estando dentro da rede da empresa.

SEM REDE

- O antigo 'castelo com muralha' não existe mais, dentro não é sinônimo de seguro.
- Cada requisição é tratada como se viesse da internet aberta.



REGRA DE VERIFICAÇÃO

Toda chamada precisa provar identidade e permissão antes de passar.

```
if (identidade_valida && permissao_ok) {  
  libera_acesso()  
}
```



LINGUAGENS

- Empresas com times remotos, nuvem e múltiplos dispositivos acessando sistemas.
- Ferramentas: IAM, MFA, VPN Zero Trust, Google BeyondCorp.



FLUXO BÁSICO

- PASSO 1: Usuário ou sistema pede acesso a um recurso
- PASSO 2: Identidade, dispositivo e contexto são verificados na hora
- PASSO 3: Acesso liberado só ao mínimo necessário, por tempo limitado



IDEIA CENTRAL



- Nuvem é como um prédio alugado: síndico cuida da estrutura, você cuida do seu apartamento.

RESUMO EXPRESSO



- Provedor protege a infra → Você protege os dados → Falha de config é sua culpa

PEGADINHAS



- Achar que 'está na nuvem' significa 'está seguro automaticamente'.
- Deixar bucket ou banco de dados público por esquecer a

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: config:
`bucket_privado = true` (você define is
- Resultado imediato no terminal



DEVER DO PROVEDOR

- Cuida da segurança 'DA nuvem': prédios, servidores, cabos, energia.
- Protege data centers, hardware e rede física.



DEVER DA SUA EMPRESA

Cuida da segurança 'NA nuvem': dados, senhas, configurações.

```
config: bucket_privado = true (você define isso!)
```



TIPOLOGIAS

- Todo serviço em nuvem: AWS, Azure, Google Cloud.
- S3, EC2, banco de dados, contas de usuário.



FLUXO BÁSICO

- PASSO 1: Provedor entrega a infraestrutura pronta e protegida
- PASSO 2: Sua empresa configura acessos, senhas e dados
- PASSO 3: Falha de configuração sua = vulnerabilidade sua, não do provedor



388. SEGURANÇA EM NUVEM: RESPONSABILIDADE COMPARTILHADA

É a divisão de tarefas de segurança: uma parte é do provedor de nuvem, outra é sua.

IDEIA CENTRAL



- Controle de acesso é dar crachás diferentes pra cada pessoa entrar em salas diferentes.

RESUMO EXPRESSO



- Papel ou Atributo → Regra de Acesso → Permitir ou Bloquear

389. IAM: RBAC VS ABAC

É a forma de decidir quem pode acessar o quê, baseado em cargo (RBAC) ou em características do contexto (ABAC).

RBAC (PAPÉIS)

- Permissão é ligada ao cargo, tipo 'gerente' ou 'estagiário'.
- Simple de configurar, mas rígido para casos especiais.

IDEIA CENTRAL



- Desastres acontecem: queda de servidor, incêndio, ataque hacker, erro humano.
- O plano garante que a empresa volte a funcionar sem quebrar.

RESUMO EXPRESSO



- Desastre acontece → Plano ativa recuperação → Empresa volta a operar sem quebrar

PEGADINHAS



- Confundir RTO (tempo parado) com RPO (dado perdido).
- Dica de ouro: quanto menor o RTO/RPO, maior o custo do backup.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: RTO = hora do desastre até sistema voltar no terminal
- Resultado imediato no terminal



390. PLANO DE CONTINUIDADE DE NEGÓCIOS E DISASTER RECOVERY (RPO E RTO)

É o plano que diz quanto tempo sua empresa aguenta ficar fora do ar e quanto dado pode perder num desastre.

RTO E RPO

- RTO: tempo máximo tolerável parado, até voltar a operar.
- RPO: quantidade máxima de dado que pode se perder, em tempo.



COMO MEDIR

RTO = hora do desastre até sistema voltar no ar.

RPO = tempo desde o último backup válido.



LINGUAGENS

- Bancos, hospitais, e-commerces que não podem parar.
- Ferramentas: backup em nuvem, servidores redundantes, replicação de dados.



FLUXO BÁSICO

- PASSO 1: Desastre acontece e sistema cai
- PASSO 2: Equipe aciona plano de recuperação
- PASSO 3: Sistema volta dentro do RTO com dados até o RPO definido



IDEIA CENTRAL



- Imagine um ladrão que troca a fechadura da sua casa e vende a chave nova.

RESUMO EXPRESSO



- Clique errado → Dados trancados → Backup imutável salva a empresa

PEGADINHAS



- Achar que pagar o resgate garante a devolução dos dados.
- Ter backup, mas conectado à mesma rede infectada.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `clique_no_anexo()` → `executa_script()` → `cripto`
- Resultado imediato no terminal



391. RANSOMWARE: ANATOMIA, DISSEMINAÇÃO E PREVENÇÃO

É um programa malicioso que tranca seus arquivos e pede pagamento para devolvê-los.

CRİPTOGRAFIA MALICIOSA

- O vírus embaralha seus arquivos com uma senha secreta só do atacante.
- Sem essa senha, os dados ficam ilegíveis para sempre.



COMO ELE SE INSTALA

Geralmente entra por e-mail falso, link ou anexo infectado.

```
clique_no_anexo() →  
executa_script() →  
criptografa_arquivos()
```



LINGUAGENS

- Ataca hospitais, empresas, governos e usuários comuns.
- Famosos: WannaCry, LockBit, REvil, Conti.



FLUXO DO ATAQUE

- PASSO 1: Vítima abre arquivo ou link infectado sem saber
- PASSO 2: Vírus criptografa arquivos e se espalha na rede
- PASSO 3: Tela de resgate exhibe cobrança em criptomoeda



IDEIA CENTRAL



- Câmeras e roteadores infectados viram 'zumbis' que obedecem a um chefe invisível.

RESUMO EXPRESSO



- Aparelho fraco → vira zumbi → ataque em massa contra terceiros

PEGADINHAS



- Achar que só computador pega vírus, mas câmera e roteador também.
- Dica de ouro: troque a senha padrão de fábrica assim que instalar.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `connect(C2_server) → aguarda_ordem() → atacar`
- Resultado imediato no terminal



392. BOTNETS E DISPOSITIVOS IOT COMPROMETIDOS

Botnet é um exército de aparelhos invadidos que um criminoso controla à distância para atacar em massa.

COMO O

- Malware entra por senha padrão ou falha não corrigida no aparelho.
- Dispositivo continua funcionando normal, mas obedece ao invasor em segredo.



CÓDIGO

O malware roda um script que conecta o aparelho a um servidor de comando.

```
connect(C2_server) →  
aguarda_ordem() →  
atacar(alvo)
```



LINGUAGENS

- Ataques DDoS que derrubam sites e serviços online inteiros.
- Casos famosos: botnet Mirai, que usou câmeras e roteadores domésticos.



FLUXO BÁSICO

- PASSO 1: Hacker escaneia a internet buscando IoT com senha fraca
- PASSO 2: Malware infecta o aparelho e ele entra na botnet
- PASSO 3: Milhares de zumbis disparam tráfego junto contra um alvo



IDEIA CENTRAL



- Imagine um carteiro que abre suas cartas, lê, e às vezes troca o conteúdo antes de entregar.

RESUMO EXPRESSO



- Mentira no ARP → Tráfego desviado → Invasor no meio da conversa

PEGADINHAS



- Achar que HTTPS sozinho impede todo tipo de MitM.
- Ignorar alertas de certificado inválido no navegador.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: ARP liga endereço IP ao endereço físico (MAC)
- Resultado imediato no terminal



393. ATAQUES MAN-IN- THE- MIDDLE (MITM) E ARP SPOOFING

É quando um invasor se coloca escondido entre duas pessoas conversando para escutar ou alterar tudo que passa.

ARP SPOOFING

- Invasor mente para a rede dizendo 'eu sou o roteador'.
- Todo tráfego passa por ele antes de seguir viagem.



TABELA ARP FALSIFICADA

ARP liga endereço IP ao endereço físico (MAC) do dispositivo.

IP_rotador ->
MAC_invasor (mentira inserida na tabela)



LINGUAGENS

- Testes de segurança em redes Wi-Fi corporativas e domésticas.
- Ferramentas: Ettercap, Bettercap, Wireshark.



FLUXO DO ATAQUE

- PASSO 1: Invasor envia mensagens ARP falsas na rede
- PASSO 2: Vítima e roteador passam a mandar dados para o invasor
- PASSO 3: Invasor lê, altera ou repassa o tráfego sem ser notado



IDEIA CENTRAL



- Em vez de invadir cada empresa, o hacker envenena a fonte que todas usam.

RESUMO EXPRESSO



- Biblioteca confiável infectada → Milhares de empresas atualizam → Todas ficam vulneráveis

PEGADINHAS



- Achar que 'biblioteca popular' significa 'biblioteca segura'.
- Dica de ouro: sempre confira a versão antes de atualizar dependências.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



394. ATAQUES DE SUPPLY CHAIN (CADEIA DE SUPRIMENTOS)

É quando o criminoso ataca um fornecedor pequeno e confiável para atingir milhares de clientes de uma vez.

BIBLIOTECA INFECTADA

- Hacker insere código malicioso dentro de um pacote open source popular.
- Milhares de projetos baixam essa 'peça' sem suspeitar de nada.



CÓDIGO

O malware fica escondido dentro de uma função normal do pacote.
npm install biblioteca-popular → vem com vírus junto



LINGUAGENS

- Gerenciadores de pacotes como npm, PyPI, Maven e NuGet.
- Casos reais: SolarWinds, event-stream, xz-utils.



FLUXO BÁSICO

- PASSO 1: Hacker ganha acesso ao código da biblioteca famosa
- PASSO 2: Insere um código malicioso disfarçado de atualização
- PASSO 3: Empresas atualizam e o vírus entra por dentro do sistema



IDEIA CENTRAL



- Imagine uma fechadura com defeito de fábrica que ninguém percebeu ainda.

RESUMO EXPRESSO



- Falha oculta → Alguém descobre antes do fabricante → Vira arma ou dinheiro

PEGADINHAS



- Achar que atualizações são chatas e não importam.
- Dica de ouro: sempre atualize seus programas, elas fecham essas falhas.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `exploit.py --target=falha_secreta --executar()`
- Resultado imediato no terminal



395. ZERO-DAY EXPLOITS E O MERCADO DE VULNERABILIDADES

É uma falha secreta em um software que nem o próprio fabricante sabe que existe.

ZERO-DAY

- O nome vem de 'zero dias' para o fabricante corrigir o problema.
- Quem descobre primeiro tem vantagem enorme sobre todo mundo.



EXPLOIT

É o código feito para explorar essa falha escondida.

```
exploit.py --  
target=falha_secreta --  
executar()
```



LINGUAGENS

- Espionagem, ataques cibernéticos e pesquisa de segurança.
- Empresas como Zerodium e governos compram esses achados.



FLUXO BÁSICO

- PASSO 1: Alguém encontra uma falha desconhecida no software
- PASSO 2: Decide vender, usar em ataque ou avisar o fabricante
- PASSO 3: Fabricante corrige com uma atualização (patch)



IDEIA CENTRAL



- É como desmontar um liquidificador para entender como ele funciona, sem manual.

RESUMO EXPRESSO



- Binário misterioso → Ghidra traduz → Você entende a lógica interna

PEGADINHAS



- Achar que o pseudocódigo é perfeito, ele pode ter erros de tradução.
- Dica de ouro: comece pela função 'main', é o ponto de partida.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `if (var1 == 5) { chamar_funcao(); }`
- Resultado imediato no terminal



396. ENGENHARIA REVERSA DE BINARIOS COM GHIDRA

É abrir um programa já pronto (sem o código original) e descobrir o que ele faz por dentro.

BINÁRIO E

- O programa compilado é só números (bytes), ilegível para humanos.
- Ghidra traduz esses números em código parecido com C.



ASSEMBLY E

Assembly é a linguagem quase-máquina, cheia de MOV, JMP, CALL.

```
if (var1 == 5) {  
    chamar_funcao();  
}
```



LINGUAGENS

- Análise de malware, achar falhas de segurança, recuperar código perdido.
- Ferramentas parecidas: IDA Pro, Radare2, Binary Ninja.



FLUXO BÁSICO

- PASSO 1: Você importa o arquivo binário para o Ghidra.
- PASSO 2: Ele analisa e transforma o código em funções legíveis.
- PASSO 3: Você renomeia variáveis e entende a lógica escondida.



IDEIA CENTRAL



- APIs são portas de entrada de dados; se mal trancadas, qualquer um invade.

RESUMO EXPRESSO



- Requisição livre → Falta de checagem → Dado vazado ou conta roubada

PEGADINHAS



- Mass Assignment: usuário envia campo 'isAdmin:true' e vira admin sozinho.
- Excesso de dados: API retorna senha e CPF sem precisar.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: Sempre valide:
`dono_do_recurso == usuario_log`
- Resultado imediato no terminal



397. SEGURANÇA EM APIS: OWASP API SECURITY TOP 10

É a lista dos erros mais perigosos que deixam APIs vulneráveis a ataques e roubo de dados.

BOLA

- Usuário troca o ID na URL e acessa dados de outra pessoa.
- Sistema não confere se o dono do token é o dono do recurso.



CÓDIGO

Sempre valide:
`dono_do_recurso == usuario_logado.`

```
if (pedido.userId !== token.userId) throw Erro403
```



LINGUAGENS

- APIs REST de bancos, apps de saúde e e-commerce.
- Node.js, Django REST, Spring Boot, Express.



FLUXO BÁSICO

- PASSO 1: Cliente envia requisição com ID de um recurso
- PASSO 2: API verifica se o token pertence ao dono do recurso
- PASSO 3: Libera dados só se a permissão for confirmada



IDEIA CENTRAL



- Wi-Fi usa senha e criptografia para embaralhar tudo que trafega pelo ar.

RESUMO EXPRESSO



- Desautenticação → Reconexão forçada → Handshake roubado ou rede clonada

PEGADINHAS



- Achar que WPA2 é totalmente seguro; ele é vulnerável a captura de handshake.
- Confundir evil twin com a rede real só porque o nome é igual.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `capturar_handshake()` → `tentar_quebrar_senha_o`
- Resultado imediato no terminal



398. SEGURANÇA WI-FI: WPA2 VS WPA3

É o cadeado que protege sua rede sem fio para estranhos não lerem nem invadirem seus dados.

WPA2 VS WPA3

- WPA2 usa um aperto de mão (handshake) que pode ser roubado e quebrado.
- WPA3 troca esse aperto de mão por um método mais forte chamado SAE.



HANDSHAKE EM AÇÃO

É a troca inicial de mensagens que gera a chave secreta da conexão.

```
capturar_handshake() →  
tentar_quebrar_senha_offline()
```



LINGUAGENS

- Testes de segurança com ferramentas como Aircrack-ng e Wireshark.
- Redes domésticas, empresas e provas de pentest autorizado.



COMO UM

- PASSO 1: Atacante cria rede falsa idêntica (evil twin)
- PASSO 2: Envia desautenticação para expulsar o cliente da rede real
- PASSO 3: Cliente reconecta e o handshake é capturado ou roubado



IDEIA CENTRAL



- A IA aprende seu rosto ou voz e cria uma versão falsa idêntica.
- É a mentira ficando indistinguível da verdade.

RESUMO EXPRESSO



- Sua voz/rosto → IA clona → Golpe parece verdadeiro e urgente

PEGADINHAS



- Achar que 'ouvir a voz' é prova suficiente de identidade.
- Ouro: sempre confirme por outro canal antes de agir com pressa.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: voz_clonada = ligação('mãe em perigo, preciso')
- Resultado imediato no terminal



399. DEEPPFAKES, IA MALICIOSA E CLONAGEM DE VOZ

É usar inteligência artificial para criar vídeos, áudios ou imagens falsas que parecem 100% reais.

COMO A IA FAZ ISSO

- Ela treina com fotos, vídeos ou áudios reais da pessoa.
- Depois gera novo conteúdo imitando padrões de voz e rosto.



ENGENHARIA SOCIAL 2.0

Antes: golpista fingia ser alguém por texto ou ligação estranha.

```
voz_clonada =  
ligação('mãe em perigo,  
preciso de PIX')
```



LINGUAGENS

- Golpes de falso sequestro e falsos chefes pedindo transferências.
- Ferramentas: ElevenLabs, DeepFaceLab, apps de



COMO O GOLPE FUNCIONA

- PASSO 1: Golpista coleta áudios ou vídeos públicos da vítima ou de um parente.
- PASSO 2: IA clona a voz ou rosto em minutos com poucos segundos de amostra.
- PASSO 3: Vítima recebe ligação ou vídeo falso e é induzida a agir com urgência



IDEIA CENTRAL



- É como um caça-tesouro: você acha a falha, a empresa te paga a recompensa.

RESUMO EXPRESSO



- Achar falha → Reportar com evidências → Receber pagamento pela empresa

PEGADINHAS



- Testar fora do escopo é ilegal, mesmo em plataforma de bug bounty.
- Relatório bem escrito vale mais que a falha em si, seja claro e objetivo.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: `escopo: *.empresa.com` (permitido) vs `app-inte`
- Resultado imediato no terminal



400. BUG BOUNTY: FATURANDO AO CAÇAR FALHAS LEGALMENTE

Bug Bounty é um programa onde empresas pagam para você achar e reportar falhas de segurança nos sistemas delas.

PLATAFORMAS INTERMEDIÁRIAS

- HackerOne e Bugcrowd conectam hackers éticos a empresas com programas abertos.
- Elas definem regras, escopo e pagam com segurança para os dois lados.



ESCOPO E REGRAS

Cada programa define o que pode e o que não pode ser testado.

escopo: `*.empresa.com`
(permitido) vs `app-interno.com` (fora)



LINGUAGENS

- Testes em sites, apps mobile, APIs e redes de empresas reais.
- Ferramentas comuns: Burp Suite, Nmap, OWASP ZAP, Postman.



FLUXO DO CAÇADOR

- PASSO 1: Escolha um programa e leia o escopo com atenção
- PASSO 2: Investigue o sistema procurando falhas dentro das regras
- PASSO 3: Reporte com provas claras e receba a recompensa



IDEIA CENTRAL



- Um lab é uma 'sala de treino' isolada só para testes e erros.

RESUMO EXPRESSO



- VM isolada → Kali ataca Metasploitable → Aprendizado sem risco

PEGADINHAS



- Deixar a VM em modo 'Bridge', expondo o ataque para a rede real.
- Dica de ouro: sempre confira o modo de rede antes de atacar.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



401. MONTANDO SEU LAB DE ESTUDOS COM KALI LINUX E METASPLOITABLE

É criar um laboratório virtual seguro para praticar hacking ético sem arriscar seu computador real.

MÁQUINA VIRTUAL

- Simula um computador dentro do seu computador real.
- Se algo quebrar, você só apaga a VM, seu PC fica intacto.



REDE ISOLADA

Configuração que impede a VM de acessar a internet real.

VirtualBox: Configurações > Rede > Modo Somente-Host



LINGUAGENS

Estudo para certificações como CEH e pentest profissional.

- Ferramentas: VirtualBox, VMware, Kali Linux, Metasploitable.



PASSO A PASSO DO LAB

- PASSO 1: Instale o VirtualBox e crie a rede isolada
- PASSO 2: Suba a VM Kali (ataque) e a Metasploitable (alvo)
- PASSO 3: Pratique ataques só dentro dessa rede fechada



IDEIA CENTRAL



- Aprender hacking é como jogar um videogame com fases e recompensas reais de conhecimento.

RESUMO EXPRESSO



- Máquina vulnerável → Invasão guiada por desafios → Habilidade real de segurança

PEGADINHAS



- Iniciante quer pular etapas e ignora a fase de reconhecimento do alvo.
- Dica de ouro: anote tudo, todo detalhe pode ser a chave da invasão.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



402. PLATAFORMAS DE TREINAMENTO: HACK THE BOX E TRYHACKME

São academias virtuais onde você invade máquinas de propósito, de forma legal, para aprender segurança na prática.

GAMIFICAÇÃO

- Pontos, rankings e badges motivam você a continuar treinando todos os dias.
- Cada máquina vulnerável é uma 'fase' com dificuldade crescente.



FLAG (A MOEDA DO JOGO)

A prova de que você invadiu é um código secreto chamado flag.
user.txt / root.txt



LINGUAGENS

- Preparação para carreira de Pentest e Segurança Ofensiva.
- Hack The Box, TryHackMe, Kali Linux, Nmap, Burp Suite.



FLUXO BÁSICO

- PASSO 1: Você escolhe uma máquina virtual vulnerável para atacar
- PASSO 2: Você escaneia, encontra falhas e ganha acesso ao sistema
- PASSO 3: Você captura a flag e sobe no ranking da plataforma



IDEIA CENTRAL



- Certificado é como um diploma que valida seu conhecimento prático em hackear e defender sistemas.

RESUMO EXPRESSO



- Teoria (Security+) → Mentalidade ofensiva (CEH) → Prática comprovada (OSCP)

PEGADINHAS



- Achar que decorar teoria substitui prática em laboratório real.
- Pular etapas e tentar OSCP sem base gera frustração e reprovação.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



403. CERTIFICAÇÕES DE CIBERSEGURANÇA: SECURITY+, CEH E OSCP

Certificações são provas que comprovam sua habilidade em segurança digital para o mercado mundial.

OS 3 NÍVEIS DO ROADMAP

- Security+ é a base teórica de segurança para todo iniciante.
- CEH ensina a pensar como um invasor de forma ética.
- OSCP prova na prática que você invade sistemas reais.



COMO SÃO AS PROVAS

Security+ e CEH são provas de múltipla escolha teóricas. OSCP é 24h invadindo máquinas reais sem gabarito.



LINGUAGENS

- Exigido em vagas de analista SOC, pentester e red team.
- Reconhecido em empresas como Google, IBM e bancos globais.



PASSO A

- PASSO 1: Tire o Security+ para entender fundamentos de redes e defesa
- PASSO 2: Faça o CEH para aprender ferramentas e mentalidade de ataque
- PASSO 3: Conquiste o OSCP e prove na prática que você é um hacker ético



IDEIA CENTRAL



- Toda imagem Docker pode esconder pacotes com falhas conhecidas.
- Escanear antes de publicar evita levar problemas para o mundo real.

RESUMO EXPRESSO



- Imagem criada → Scanner verifica pacotes → Só vai pra produção se estiver segura

PEGADINHAS



- Ignorar vulnerabilidades 'médias' que se acumulam e viram porta de entrada.
- Rodar Docker Bench Security no host, não na

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



É como revistar as malas de um viajante antes de embarcar, garantindo que nada perigoso entre em produção.

404. SEGURANÇA EM CONTÊINERES: TRIVY E DOCKER BENCH SECURITY

TRIVY

- Lê camadas da imagem e compara pacotes com bancos de vulnerabilidades.
- Aponta nível de risco: baixo, médio, alto ou crítico.



CÓDIGO

Comando roda o scanner apontando para a imagem criada.
trivy image minha-app:1.0



LINGUAGENS

- Dentro de pipelines de CI/CD antes do deploy final.
- GitHub Actions, GitLab CI, Jenkins e ambientes Kubernetes.



FLUXO BÁSICO

- PASSO 1: Build gera a imagem Docker com suas dependências.
- PASSO 2: Trivy escaneia pacotes e o Bench Security audita configurações do host.
- PASSO 3: Pipeline bloqueia o deploy se achar falha crítica.



IDEIA CENTRAL



- Container não é uma caixa mágica isolada, ele pode escapar e tocar o servidor real.

RESUMO EXPRESSO



- Container pede acesso → Regra Restricted barra root/host → Cluster protegido

PEGADINHAS



- Achar que 'isolado' significa 'seguro', não é a mesma coisa.
- Esquecer hostPath e hostNetwork, portas de fuga pro node.

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



São as regras que travam containers para não virarem 'chave mestra' da máquina.

405. SEGURANÇA EM KUBERNETES: POD SECURITY STANDARDS E RBAC

POD SECURITY STANDARDS

- São 3 níveis de rigor: Privileged, Baseline e Restricted.
- Restricted bloqueia root e acesso ao host do node.



CÓDIGO

No YAML do Pod, define o comportamento permitido.
runAsNonRoot: true /
privileged: false



LINGUAGENS

- Times de DevSecOps protegendo clusters em produção.
- Ferramentas: kubectl, OPA/Gatekeeper, Kyverno.



FLUXO BÁSICO

- PASSO 1: Pod pede permissão para rodar no cluster
- PASSO 2: Kubernetes checa Pod Security Standard e RBAC do usuário
- PASSO 3: Se violar regra (ex: rodar como root), Pod é bloqueado



IDEIA CENTRAL



- Active Directory é a lista de todos os usuários e computadores da empresa.

RESUMO EXPRESSO



- Login → Bilhete Kerberos → Acesso liberado na floresta AD

PEGADINHAS



- Confundir domínio com floresta: floresta pode ter vários domínios.
- Golden Ticket: invasor forja um bilhete mestre e vira admin.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: TGT = bilhete geral de acesso ao domínio.
- Resultado imediato no terminal



406. SEGURANÇA EM AD É KERBEROS

É o sistema que verifica quem você é e o que pode acessar dentro de uma empresa.

KERBEROS

- Sistema de bilhetes que prova sua identidade sem repetir a senha.
- Cada acesso gera um tíquete temporário e criptografado.



FLUXO DO TÍQUETE

TGT = bilhete geral de acesso ao domínio.

TGS(TGT) → acesso a serviço específico



LINGUAGENS

- Login em computadores de empresas e servidores internos.
- Ferramentas: Windows Server, PowerShell, BloodHound



COMO O LOGIN FUNCIONA

- PASSO 1: Você digita usuário e senha no computador
- PASSO 2: O KDC confere e entrega um bilhete (TGT)
- PASSO 3: Esse bilhete libera acesso aos sistemas da empresa



IDEIA CENTRAL



- Você não precisa saber a senha real, só uma cópia criptografada dela guardada na memória.

RESUMO EXPRESSO



- Rouba hash → Reutiliza como senha → Domina a rede inteira

PEGADINHAS



- Achar que trocar a senha resolve: o Golden Ticket continua válido até resetar o KRBTGT duas vezes.
- Achar que só quem sabe a senha em texto puro é

EXEMPLO PRÁTICO

- Entrada: dados ou solicitação
- Regra: processamento lógico
- Saída: solução esperada



407. PASS-THE-HASH E GOLDEN TICKET NO ACTIVE DIRECTORY

Invasores roubam a 'senha disfarçada' (hash) ou forjam um crachá mestre para virar administrador da rede.

COMO O

- O Windows guarda hashes de senha na memória do processo LSASS.
- Quem tem acesso de admin na máquina pode extrair esses hashes.



FERRAMENTA E SINTAXE

Ferramentas como Mimikatz extraem e reutilizam o hash direto.
`sekurlsa::pth /user:admin /domain:corp /ntlm:HASH`



LINGUAGENS

- Testes de invasão (pentest) e simulações de ataque real (Red Team).
- Ferramentas: Mimikatz, Impacket, Rubeus, PowerSploit.



FLUXO DO ATAQUE

- PASSO 1: Invasor compromete um computador e rouba o hash da memória
- PASSO 2: Usa o hash para se autenticar como se fosse a pessoa dona dele
- PASSO 3: Com o hash do KRBTGT, forja um 'Golden Ticket' e vira dono do domínio



IDEIA CENTRAL



- É como ter um assistente que bate na porta de cada casa da rua para ver quem responde.

RESUMO EXPRESSO



- Monta pacote/pedido → Envia e espera resposta → Analisa resultado da rede

PEGADINHAS



- Testar redes sem permissão é ilegal, sempre peça autorização antes.
- Esquecer de tratar erros faz o script travar quando não há

EXEMPLO PRÁTICO

- Aplicação prática:
- Código:
`sr1(IP(dst='192.168.0.1')/ICMP())`
- Resultado imediato no terminal



408. AUTOMAÇÃO DE SEGURANÇA COM PYTHON (SCAPY E REQUESTS)

Usar Python para criar robôs que testam e verificam a segurança de uma rede automaticamente.

SCAPY E REQUESTS

- Scapy cria e envia pacotes de rede, como cartas personalizadas.
- Requests conversa com sites, como fazer uma pergunta e ouvir a resposta.



CÓDIGO

Scapy monta o pacote antes de enviar.

```
sr1(IP(dst='192.168.0.1')/ICMP())  
requests.get('http://site.com')
```

LINGUAGENS

- Verificar se um computador da rede está ligado e respondendo.
- Testar se um site aceita conexões seguras ou tem falhas simples.



FLUXO BÁSICO

- PASSO 1: O script monta um pacote ou pedido de teste
- PASSO 2: Ele envia para o alvo e espera a resposta
- PASSO 3: O script analisa a resposta e mostra o resultado



IDEIA CENTRAL



- Diretor não entende 'XSS', mas entende 'prejuízo de R\$ 2 milhões'.

RESUMO EXPRESSO



- Falha técnica → Tradução em dinheiro e reputação → Decisão da diretoria.

PEGADINHAS



- Erro: falar de CVSS e jargão técnico sem mostrar o custo real.
- Dica de ouro: comece pelo prejuízo, não pela falha.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: Risco = Chance do ataque acontecer x Custo do
- Resultado imediato no terminal



409 RELATÓRIO DE PENTEST: DA FALHA TECNICA AO, PREJUÍZO REAL

É transformar uma falha de segurança em um número de risco financeiro que a diretoria entende.

TRADUÇÃO DE RISCO

- Cada falha técnica ganha um valor: multa, prejuízo ou parada do sistema.
- Severidade técnica (alta/média/baixa) vira prioridade de investimento.



FÓRMULA DO RISCO

Risco = Chance do ataque acontecer x Custo do estrago.

risco = probabilidade * impacto_financeiro



LINGUAGENS

- Reuniões de board, auditorias e definição de orçamento de segurança.
- Bancos, e-commerce, hospitais e seguradoras.



FLUXO DO RELATÓRIO

- PASSO 1: Time técnico encontra a falha e mede sua gravidade.
- PASSO 2: Especialista converte a falha em prejuízo, multa ou parada.
- PASSO 3: Diretoria decide investir em correção ou aceitar o risco.



IDEIA CENTRAL



- Os cadeados digitais de hoje podem ser quebrados por computadores do futuro.

RESUMO EXPRESSO



- Computador quântico → quebra RSA → PQC entra para proteger o futuro.

PEGADINHAS



- Achar que a ameaça é só teórica: dados roubados hoje podem ser descriptografados depois.
- Dica de ouro: priorize migrar dados de longa duração.

EXEMPLO PRÁTICO

- Aplicação prática:
- Código: Baseiam-se em problemas de reticulados (lattices)
- Resultado imediato no terminal



410. O FUTURO DA CIBERSEGURANÇA: CRIPTOGRAFIA POS-QUÂNTICA

São novas fórmulas matemáticas criadas para proteger seus dados mesmo contra computadores quânticos.

A AMEAÇA

- RSA protege dados fatorando números gigantes, algo quase impossível hoje.
- Computador quântico usa o Algoritmo de Shor e fatora esses números rapidamente.



NOVOS ALGORITMOS PQC

Baseiam-se em problemas de reticulados (lattices), difíceis até para quânticos.

Ex: ML-KEM (Kyber) para chaves, ML-DSA



LINGUAGENS

- Navegadores, bancos e governos já testam PQC em conexões HTTPS.
- Padronizado pelo NIST dos EUA, adotado por Google, Cloudflare e Apple



FLUXO DA MIGRAÇÃO

- PASSO 1: Sistema identifica onde usa criptografia vulnerável (RSA, ECC)
- PASSO 2: Troca o algoritmo antigo por um algoritmo pós-quântico aprovado
- PASSO 3: Dados ficam protegidos mesmo contra ataques quânticos futuros

